

Hardware-Accelerated Transformer Attention for Intelligent Edge Vision: Architectural Exploration and FPGA Deployment on AMD Kria KV260

Trần Duy Tuấn*, Thái Hoàng Gia Thoại†

Department of Electrical and Electronics Engineering

Ho Chi Minh City University of Technology (HCMUT), VNU-HCM

Student IDs: *2353278, †2353146

I. INVESTIGATION FRAMEWORK

When the high-level overview, phase roadmap, tech stack, and constraints are already provided, the investigation phase is about **de-risking the project, refining ambiguous requirements**, and **validating technical feasibility** before committing to full-scale development.

The investigation follows a six-stage operational blueprint for systematically evaluating the capstone project from inception.

1) *1Deconstruct Scope and Map Constraints*: Establish operational boundaries before writing code

Start by translating the provided overview into formal specifications to ensure team and advisor alignment.

- **Classify Requirements**: Group features into **Must-Have**, **Should-Have**, **Could-Have**, and **Won't-Have** (MoSCoW framework) to protect against scope creep.
- **Audit Fixed Constraints**: Document hard limits across time (submission deadlines), budget (cloud credits, hardware costs), latency/compute requirements, and data privacy policies (e.g., GDPR, HIPAA).
- **Define Quantitative Success Metrics**: Establish clear acceptance criteria (e.g., *inference latency < 150 ms, F1-score > 0.88, system handles 500 concurrent WebSocket connections*).
- **Deliverable**: A single-page **Project Charter & Constraint Matrix**.

2) *2Conduct Prior Art and Industry Benchmarking*: Avoid reinventing existing solutions

Analyze how similar academic or open-source projects solved identical problems.

- **Literature & Patent Scan**: Review 5–8 recent papers (via arXiv, IEEE Xplore, Google Scholar) published within the last 2–3 years to identify current baseline algorithms.
- **Open-Source Teardowns**: Inspect GitHub repositories of related systems. Evaluate their repository structure, chosen database models, and bottlenecks reported in open issue trackers.
- **Competitor / State-of-the-Art (SOTA) Gap Analysis**: Document where the project builds on, optimizes, or departs from existing implementations.

- **Deliverable**: A **Literature & Benchmark Matrix** summarizing related approaches, inputs/outputs, advantages, and known limitations.

3) *3Audit Data, External Dependencies, and APIs*: Identify third-party failure points early

Identify data inputs, third-party APIs, and external tooling prerequisites before designing system flows.

- **Data Pipeline Assessment**: If training or consuming datasets, verify licensing, availability, missing data rates, class balance, and preprocessing pipelines needed.
- **API & SDK Stress Testing**: Inspect rate limits, pricing tiers, uptime SLAs, and authentication flows (OAuth2, JWT) for external services.
- **Fallback Strategy**: Plan local mocks (e.g., synthetic datasets, mock API servers with MSW or WireMock) so work can continue during offline testing or quota limits.
- **Deliverable**: A **Data & Dependency Catalog** with rate limits, licensing, and schema snapshots.

4) *4Execute Targeted Technical Spikes (PoCs)*: Validate risky assumptions with throwaway code

Create minimal proofs-of-concept (PoCs) focused entirely on the highest-uncertainty areas of the mandated tech stack.

- **Identify the Riskiest Unknowns**: Focus on non-trivial integrations (e.g., real-time WebRTC streams, complex ML inference pipelines, microservice IPC).
- **Timeboxed Spikes**: Dedicate isolated 24–48 hour timeboxes to build throwaway scripts testing whether *Component A* can reliably communicate with *Component B*.
- **Performance Benchmarking**: Measure memory footprints, network serialization overhead, and CPU/GPU utilization during peak load spikes.
- **Deliverable**: 2–3 **Proof-of-Concept Scripts** and a brief **Spike Evaluation Report** validating core feasibility.

5) *5Draft the System Architecture and Interface Contracts*: Define data contracts before building modules in parallel

With technical viability confirmed, architect the system components and establish integration points.

- **High-Level Architecture Diagrams**: Diagram components (C4 Model - Context, Container, Component levels),

illustrating data flows, message brokers, caching layers, and persistence storage.

- **Schema & API First Design:** Define formal OpenAPI / Swagger specs, GraphQL schemas, or Protobuf definitions. This enables frontend, backend, and data members to work in parallel without blocking each other.
- **Database Entity-Relationship Diagram (ERD):** Outline relational schemas or NoSQL document structures, including primary/foreign keys, indexes, and partition keys.
- **Deliverable: An Architecture Design Document (ADD / RFC)** containing interface schemas and architectural blueprints.

6) 6Synthesize the Risk Registry and Execution Roadmap:

Convert research findings into structured sprints

Finalize the investigation by translating findings into an agile execution plan with proactive mitigations.

- **Create a Failure Mode & Effects Matrix:** Identify potential project pitfalls (e.g., model underperformance, free tier quota exhaustion, API deprecation) and specify fallback plans for each.
- **Sprint Breakdown:** Divide the semester or timeline into 2-week milestones with tangible deliverables at every checkpoint.
- **Verification Strategy:** Define unit, integration, and user evaluation protocols to prove the system fulfills the success criteria defined in Step 1.
- **Deliverable: A Work Breakdown Structure (WBS) and Risk Mitigation Log** tracked in a project board (Jira, Linear, or GitHub Projects).

A. Investigation Deliverables Checklist

II. PROJECT-SPECIFIC INVESTIGATION BLUEPRINT

The six-stage investigation blueprint is applied to **Hardware-Accelerated Transformer Attention for Intelligent Edge Vision** using a Kria KV260 and SystemVerilog workflow.

1) *1Deconstruct Scope and Map Constraints:* Lock down the hardware-software boundary

For an FPGA-accelerated ViT, the most critical decision is partitioning which operations run on the ARM Processing System (PS) and which run on the Programmable Logic (PL).

- **Must-Haves (Phase 1):** Python/PyTorch software baseline, custom SystemVerilog datapath for

$$\text{Attention}(Q, K, V) = \text{softmax}\left(\frac{QK^T}{\sqrt{d_k}}\right)V. \quad (1)$$

, one hardware softmax approximation, AXI DMA integration, and functional deployment on the KV260.

- **Should-Haves (Phase 2):** Second softmax implementation, parameterized sequence lengths/head dimensions, hardware performance counters.
- **Constraints:** The Kria KV260 has strict limits on DSP slices (252) and BRAM (144 blocks). The fixed-point precision (e.g., INT8 vs. INT16) must balance accuracy with these resource limits.
- **Metrics:** Measure End-to-end latency (ms), PL throughput (GOPS), numerical error (Mean Squared Error against

FP32 PyTorch), and resource utilization (LUT, FF, DSP, BRAM).

2) *2Conduct Prior Art and Industry Benchmarking:* Target hardware-friendly approximations

Standard floating-point softmax requires expensive exponential and division operations. Prior-art investigation must therefore examine how existing RTL implementations reduce or replace these operations.

- **Softmax Approximations:** Research LUT-based softmax (storing pre-computed exponentials in BRAM) versus piecewise-linear approximation (e.g., breaking the exponential curve into base-2 shifts and additions).
- **ViT Accelerators:** Look into papers like *SpAtten* or *VAIM* to see how they handle QK multiplication without blowing up the DSP count.
- **Model Selection:** Investigate lightweight vision models like MobileViT, TinyViT, or LeViT. Standard ViT-Base is too large for edge FPGA BRAM.

3) *3Audit Data, External Dependencies, and APIs:* Prepare the data flow and test vectors

Before touching Vivado, ensure the Python environment can generate the exact stimuli needed for SystemVerilog testbenches.

- **Dataset:** Pick a standard, lightweight dataset for the baseline. For occupancy/activity, consider UCF101 or a subset of COCO.
- **Golden Reference Generation:** Write a PyTorch script that extracts the intermediate FP32 tensors (Q, K, V inputs) from the chosen Transformer layer and saves them as .hex or .txt files.
- **Toolchain Audit:** Verify the Vivado 2023.x (or the current version) board files for the Kria KV260 are installed, and confirm the AXI DMA IP licensing is active.

4) *4Execute Targeted Technical Spikes (PoCs):* De-risk the math and memory movement

Run short, focused experiments on the highest-risk areas before designing the full datapath.

- **Quantization Spike (Python):** Simulate fixed-point math in PyTorch. Truncate the FP32 tensors to INT8/INT16, run the attention calculation, and check if the model's accuracy collapses. This determines the required hardware precision.
- **RTL Math Spike (SystemVerilog):** Write a simple SystemVerilog module for a parameterized $N \times N$ fixed-point MAC (Multiply-Accumulate) unit. Run it in Vivado Simulator to verify the truncation logic.
- **DMA Loopback Spike (Hardware):** Set up a basic Block Design in Vivado with the Zynq UltraScale+ MPSoC and an AXI DMA block. Send a dummy array from Python/Jupyter on the KV260 to the PL and back to verify memory-mapped and streaming interface bandwidth.

5) *5Draft the System Architecture and Interface Contracts:* Define the AXI interfaces and block design

Map out the hardware architecture and how Tuan and Thoi will divide the module development.

TABLE I: Technical comparison: Phase; Core Deliverable; Primary Evaluation Question.

Phase	Core Deliverable	Primary Evaluation Question
Scope & Constraints	MoSCoW & Constraint Matrix	<i>Are the boundaries and quantitative targets clear to all stakeholders?</i>
Prior Art	Benchmark Matrix	<i>What makes this solution unique or optimized compared to existing work?</i>
Dependencies	Data/API Audit Catalog	<i>Are all third-party dependencies reliable, licensed, and rate-friendly?</i>
Spikes & PoCs	Spike Validation Report	<i>Did the code prove the hardest technical challenges are solvable?</i>
Architecture	C4 Diagrams & API Contracts	<i>Can team members develop separate modules independently without breaking interfaces?</i>
Planning	Risk Log & 2-Week Sprint WBS	<i>What is the exact fallback plan if a core library or model fails?</i>

- **System Architecture:** The PS (ARM cortex) captures the image, runs the initial convolutions/patch embeddings, and streams the Q, K, V matrices via AXI4-Stream to the PL. The PL calculates the attention scores, aggregates the values, and streams the output back.
- **Hardware Interface Contract:** Define the exact bit-widths of the AXI-Stream interfaces (e.g., TDATA width = 64 bits/128 bits depending on the chosen quantization and parallelization).
- **Internal Datapath:** Diagram the pipeline stages. QK multiplication $\rightarrow$ Scaling (shift operation) $\rightarrow$ Softmax (LUT/Piecewise) $\rightarrow$ Score-Value multiplication.

6) 6Synthesize the Risk Registry and Execution Roadmap:

Map tasks across Phase 1 and Phase 2

Translate the project phases into actionable development tasks and identify synthesis risks.

- **Timing Risk:** Large multiplier arrays for QK^T often cause setup and hold time violations. *Mitigation:* Plan for deep pipelining (inserting register stages between MAC operations).
- **Resource Risk:** Storing the full $N \times N$ attention matrix might exceed BRAM capacity for large sequence lengths. *Mitigation:* Implement tiling/blocking (calculating attention in smaller chunks).
- **Workload Split:** One partner can own the PyTorch quantization, golden reference generation, and PS-side integration, while the other owns the SystemVerilog datapath, Vivado block design, and AXI interfaces.

III. REQUIRED KNOWLEDGE DOMAINS

Research, design, and implementation of a **Hardware-Accelerated Vision Transformer Attention Engine** on an FPGA require the intersection of **Digital RTL Design, Computer Architecture (HW/SW Co-Design), Deep Learning, and Hardware Arithmetic**.

The following knowledge fields are organized by academic subject, key module, and direct application to the capstone.

1) 1. *Digital Logic Design & RTL Engineering:* This is the foundation for building the custom hardware datapath in SystemVerilog and meeting FPGA timing constraints.

- **Pipelining & Retiming:** Breaking complex arithmetic operations (like multi-stage Multiply-Accumulate arrays)

across multiple clock cycles to prevent setup/hold violations and maintain a high clock frequency (F_{max}).

- **Finite State Machines (FSM) & Control Logic:** Designing Mealy/Moore FSMs to orchestrate matrix streaming, manage pipeline priming/flushing, and handle handshake signals.
- **Memory Architectures on FPGA (BRAM / Distributed RAM):** Implementing dual-port block RAMs, ping-pong (double-buffering) memory schemes, and FIFO buffers to store intermediate activation tensors (Q, K, V , and attention score matrices) without stalling the datapath.
- **Hardware Verification (SystemVerilog Testbenches):** Writing self-checking testbenches, generating golden test vectors via file I/O (\$readmemh), and calculating cycle latency and throughput in Vivado Simulator.

2) 2. *Computer Architecture & Embedded HW/SW Co-Design:* This area covers how the Processing System (ARM PS) and Programmable Logic (FPGA PL) communicate on the AMD Zynq UltraScale+ (Kria KV260).

3) 3. *Deep Learning & Vision Transformer (ViT) Architectures:* Understanding the mathematical structure of Transformers is essential for deciding what to accelerate in hardware versus what to keep in software.

- **Scaled Dot-Product Attention:** Deeply understanding the four-step sequence:

$$\text{Attention}(Q, K, V) = \text{softmax}\left(\frac{QK^T}{\sqrt{d_k}}\right)V. \quad (2)$$

(Matrix Multiplication 1 $\rightarrow$ Scaling factor $\rightarrow$ Row-wise Softmax $\rightarrow$ Matrix Multiplication 2).

- **Lightweight ViT Backbones:** Profiling compact edge models (e.g., MobileViT, TinyViT, LeViT, EfficientViT) to understand patch tokenization, multi-head attention (MHA) splits, and head dimensions (d_k).
- **PyTorch Quantization & Fixed-Point Simulation:** Writing custom PyTorch inference hooks to simulate bit-width reduction (INT8 / INT16 / Q-format), quantifying accuracy degradation (Top-1 accuracy, mAP) before committing to RTL.

4) 4. *Computer Arithmetic & Hardware Approximation Methods:* Standard floating-point operators (especially exponents and divisions) are too costly for edge FPGAs. This field focuses on hardware-friendly numerical approximations.

TABLE II: Technical comparison: Core Section; What to Learn; Direct Capstone Application.

Core Section	What to Learn	Direct Capstone Application
Bus Protocols	AXI4-Lite, AXI4-Full, AXI4-Stream	Standardizing register control (AXI-Lite) and high-throughput tensor streaming (AXI-Stream).
Direct Memory Access (DMA)	AXI DMA IP, Scatter-Gather vs. Simple Mode	Offloading high-speed memory transfers of image patches/embeddings between DDR memory and PL without loading the CPU.
Memory Hierarchy & Tiling	Cache lines, memory bandwidth, spatial/temporal locality	Designing tiling/blocking algorithms so large sequence lengths fit within limited on-chip BRAM.
Driver & Host Runtime	Linux UIO (Userspace I/O), PYNQ framework, or C/C++ bare-metal/Linux drivers	Writing the host script on the ARM core to configure the DMA, trigger the hardware accelerator, and read back the results.

- **Fixed-Point Representation (Q-Format):** Defining word length, integer bits, fractional bits ($Qm.n$), and handling dynamic range, saturation arithmetic, and rounding/truncation modes to prevent overflow during matrix accumulation.
- **Hardware-Efficient Softmax Approximations:**
- **Lookup Table (LUT) Method:** Pre-calculating exponential curves and storing them in BRAM/ROM.
- **Piecewise Linear (PWL) / Base-2 Approximations:** Replacing e^x with $2^{x \log_2(e)}$ and approximating through bit-shifts and simple additions.
- **Online / Streaming Softmax:** Computing normalization factors incrementally across a row to avoid storing full attention maps in BRAM (e.g., FlashAttention-style principles).
- **DSP Optimization:** Mapping fixed-point multiplications efficiently into the dedicated DSP48E2 slices of the Kria KV260 architecture.

A. Recommended Learning & Preparation Roadmap

```

1 Step 1: PyTorch Baseline & Fixed-Point Simulation
2 +-- Profile lightweight ViT -> Extract Q, K, V tensors ->
   Simulate fixed-point math -> Measure accuracy drop.
3
4 Step 2: Micro-Architecture & SystemVerilog Datapath
5 +-- Design parameterized MAC arrays -> Implement Softmax
   approximation (LUT/PWL) -> Testbench verification.
6
7 Step 3: SoC Integration & System Evaluation (KV260)
8 +-- Package IP with AXI4-Stream -> Connect AXI DMA in
   Vivado -> Write host driver -> Benchmark FPS, latency &
   power.
    
```

Listing 1: Source code, configuration, command sequence, or directory structure transcribed verbatim.

IV. SYSTEM ARCHITECTURE

The architecture of a Hardware-Accelerated Vision Transformer on a heterogeneous SoC such as the AMD Kria KV260 requires a strict boundary between the Processing System (PS) and the Programmable Logic (PL). The ARM Cortex-A53 (PS) handles dynamic, memory-intensive, and non-linear control tasks (image decoding, patch embedding, Layer Normalization, and MLP blocks). The FPGA fabric (PL) acts as a highly parallelized coprocessor dedicated to the $\mathcal{O}(N^2)$ matrix multiplications and the non-linear softmax bottleneck of the Multi-Head Attention (MHA) mechanism.

1) *1. System-Level Component Architecture:* The interface between software and hardware relies on the Advanced eXtensible Interface (AXI) standard. Software configures the accelerator via memory-mapped **AXI4-Lite**, while bulk tensor data (Q, K, V) is streamed continuously via **AXI4-Stream** using Direct Memory Access (DMA).

2) *2. Hardware Datapath Microarchitecture (The Attention Core):* Inside the CUSTOM ViT ACCELERATOR, the datapath is organized as a deep, feed-forward pipeline. Because standard ViT attention requires the entire row of scores to compute the softmax denominator, the architecture must heavily utilize on-chip Block RAM (BRAM) to buffer intermediate results.

The data representation throughout this pipeline must be quantized (e.g., INT8 for activations, INT16/INT32 for accumulators) to maximize the use of the KV260's DSP48E2 slices.

3) *3. Detailed Component Specifications: A. Input Buffering & Tiling (BRAM Banks)* The sequence length (e.g., $N=196$ for a 14×14 patch grid) dictates BRAM usage. Because Q and K matrices cannot be processed infinitely in parallel, the design implements **tiling**. A subset of Q (e.g., $T_q \times d_k$) and K (e.g., $d_k \times T_k$) is loaded from the AXI Rx FIFO into true dual-port BRAMs. This allows the MAC array to read operands on every clock cycle without stalling waiting for AXI-Stream TVALID.

B. QK^T MAC Array This is the primary compute engine. It utilizes the KV260's DSP48E2 slices configured as Multiply-Accumulate units. To maximize throughput, the architecture uses an unrolled array where multiple dimensions of d_k are multiplied and accumulated simultaneously (spatial parallelism), outputting one or more raw attention scores per clock cycle.

C. Scaler Unit In floating-point math, scores are divided by d_k . In hardware, division is extremely expensive. By constraining d_k to specific hardware-friendly dimensions (or utilizing quantization-aware training), this block is reduced to a simple arithmetic right-shift (ASR) operation, effectively dividing by a power of two with zero DSP cost and exactly 1 clock cycle of latency.

D. Hardware Softmax Engine (The Bottleneck) The Softmax unit, defined mathematically as $e^{x_i} / \sum_j e^{x_j}$, is decomposed to avoid floating-point math:

- 1) **Find Max:** Hardware tracks the maximum value in the row (x_{max}) on the fly to prevent arithmetic overflow:

second MAC array, multiplying the $(1 \times N)$ score vector by the $(N \times d_k)$ Value matrix. This produces the final attended output vector for that specific Query, which is pushed to the AXI Tx FIFO.

4) *4. Hardware/Software Execution Flow:* The sequence diagram illustrates the strict synchronization required to prevent buffer under-runs or deadlocks between the ARM processor and the FPGA fabric.

5) *5. Managing Concurrency and Throughput:* To achieve high utilization (measured in GOPS), the architecture implements **latency hiding** through pipeline registers and double-buffering.

While the ARM PS is computing the MLP and LayerNorm of Layer $L-1$, it simultaneously initiates the DMA transfer for Layer L 's Q, K, and V tensors. On the PL side, BRAM banks are divided into "Ping" and "Pong" halves. The DMA writes the next tile of data into the Pong buffer while the DSP arrays actively compute attention on the Ping buffer. When the MAC arrays finish, the buffer pointers swap in a single clock cycle. This ensures the computationally expensive DSP48E2 slices are never idle waiting for memory transfers over the AXI bus, fulfilling the performance requirements necessary for real-time edge vision applications.

V. RECOMMENDED TOOLCHAIN

The toolchain for **Hardware-Accelerated Transformer Attention on AMD Kria KV260 using SystemVerilog** is categorized by project phase, with each tool assigned a specific engineering role.

1) *1. Toolchain Recommendation Summary:*

2) *2. Deep Dive by Development Phase:*

a) *A. Model Development, Quantization & Test Vector Generation:*

- **Google Colab / Kaggle:**
- **Best for:** Downloading datasets, fine-tuning lightweight ViT backbones (e.g., MobileViT, TinyViT), and running floating-point (FP32) training scripts on free cloud GPUs (T4/A100).
- **Local Python (PyTorch + NumPy):**
- **Best for:** Simulating fixed-point quantization (INT8/INT16) and generating intermediate tensors (Q, K, V , scaling values, softmax outputs).
- **Export Script:** Write a script to export matrices to .mem or .hex files formatted for SystemVerilog \$readmemh / \$readmemb.

b) *B. SystemVerilog RTL Design & Fast Simulation:*

- **Visual Studio Code (Primary Editor):**
- **Recommended Extensions:** **Verilog-HDL/SystemVerilog/Bluespec** **SystemVerilog** (by mshr-h), **SystemVerilog - Language Support** (by EFCoders), and **WaveTrace** or **Waveterm** for viewing .vcd files directly in VS Code.
- **Simulation Tool Comparison:**
- **Vivado Simulator (xsim): Primary choice.** Full native support for SystemVerilog (interfaces, structs, pack-

ages, multi-dimensional arrays) and Xilinx primitives (DSP48E2, True Dual-Port BRAM IPs).

- **Verilator (C++ Cycle-Accurate Sim):** Extremely fast for large batch verification and co-simulating C++/PyTorch test vectors with RTL.
- **Icarus Verilog (iverilog):** *Not recommended* for this project. While lightweight, Icarus has limited support for advanced SystemVerilog constructs (e.g., streaming operators, parameterized interfaces, complex structs).
- c) *C. FPGA Implementation & System Integration:*
- **AMD Vivado Design Suite (ML Standard Edition):**
- **Role:** Core synthesis, placement, routing, and timing analysis.
- **Vivado IP Integrator (IPI):** Essential for building the block design connecting the **Zynq UltraScale+ MPSoC IP**, **AXI Direct Memory Access (AXI DMA) IP**, and the packaged **SystemVerilog Attention Core IP**.
- **What about Vitis HLS?**

- The project brief specifies developing a custom hardware datapath in **SystemVerilog**. While Vitis HLS can synthesize C++ into RTL, writing native SystemVerilog provides direct cycle-level control over DSP multiplier unrolling, pipeline registers, and BRAM ping-pong switching. HLS is appropriate only as a fast high-level reference implementation for benchmarking against the custom RTL.

d) *D. Edge Deployment on Kria KV260:*

- **PYNQ on Kria Ubuntu:**
- The Kria KV260 runs an official Ubuntu / PYNQ Linux image. PYNQ provides high-level Python bindings (pynq.allocate for contiguous DMA memory buffers and pynq.Overlay for bitstream loading).
- This permits implementation of the host inference loop in Python on the KV260 ARM core, with test images streamed to the PL through DMA using a compact software interface.

```

1 vit-attention-kv260/
2 +-- doc/                                # Architecture specs &
   | memory maps
3 +-- host/                               # PS-side software
   | +-- model/                           # PyTorch ViT definition &
   |   | weights
4 |   +-- quant/                           # Fixed-point simulation &
   |     golden vector generator
5 |   +-- pynq_driver/                     # PYNQ runtime scripts for
   |     KV260 deployment
6 +-- rtl/                                # SystemVerilog source
   | files
7 |   +-- mac_array.sv                     # QK & Score-V MAC datapath
8 |   +-- softmax_lut.sv                   # Hardware softmax engine
9 |   +-- attention_core.sv                # Top-level datapath
10 |   +-- axis_wrapper.sv                 # AXI4-Stream & AXI-Lite
   | wrapper
11 +-- tb/                                # Verification
   |   +-- tb_attention_core.sv            # Self-checking
   |     SystemVerilog testbench
12 |   +-- test_vectors/                   # Generated .hex / .mem
   |     files
13 +-- vivado/                             # Vivado project scripts (
   |   Tcl-based)

```

Listing 2: Source code, configuration, command sequence, or directory structure transcribed verbatim.

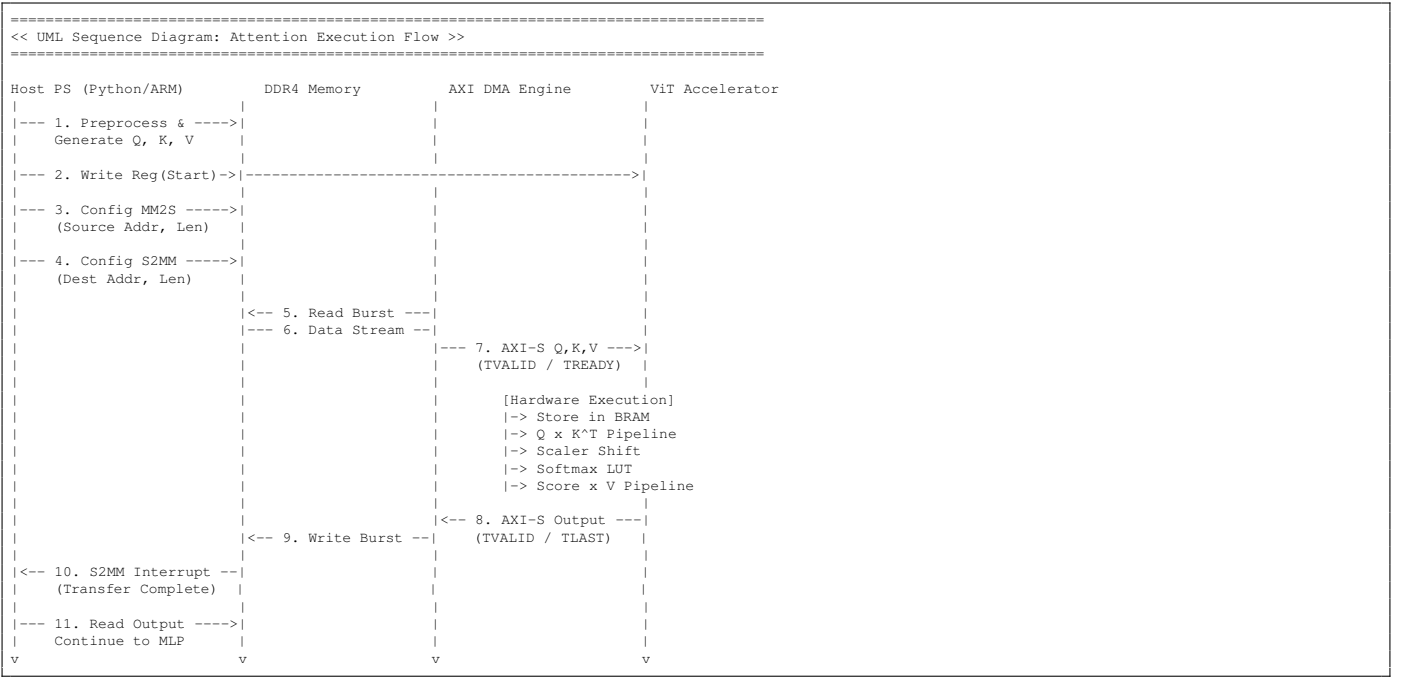


Fig. 3: Uml Sequence Diagram: Attention Execution Flow.

TABLE III: Technical comparison: Category; Recommended Tool; Alternative / Secondary; Verdict / Role.

Category	Recommended Tool	Alternative / Secondary	Verdict / Role
ML & Quantization	Local Python / VS Code	Google Colab / Kaggle	Use Colab for initial heavy GPU training; use local Python for batch golden-vector generation.
RTL Code Editor	VS Code (with SV plugins)	Neovim / Sublime	Primary editor with syntax highlighting, linters, and Git integration.
Fast RTL Simulation	Vivado Simulator (xsim) / Verilator	Icarus Verilog (iverilog)	Avoid Icarus for modern SystemVerilog. Use Vivado xsim or Verilator.
FPGA Synthesis & SoC	AMD Vivado ML Standard	—	Mandatory for block design, AXI DMA integration, synthesis, timing closure, and bitstream generation.
C/C++ HLS Tool	<i>Reference/Exploratory only</i>	Vitis HLS	Project requires SystemVerilog RTL , but HLS can be used to generate quick algorithmic reference baselines.
KV260 Runtime	PYNQ Framework	C++ / Linux UIO	PYNQ (Python on Kria Ubuntu) drastically simplifies AXI DMA buffer allocation and PS-PL benchmarking.

VI. VENDOR AND CUSTOM IP PARTITIONING

Standard SoC infrastructure IP blocks do not need to be designed from scratch.

For a platform like the **AMD Kria KV260 (Zynq UltraScale+ MPSoC)**, designing infrastructure blocks (such as the CPU, DDR memory controllers, interconnects, or DMA engines) from scratch is practically impossible and outside the scope of the project.

Vivado’s built-in vendor IP library supplies the SoC infrastructure, while the **custom Attention Accelerator IP** is designed in SystemVerilog.

1) *Build-versus-Reuse Boundary:*

2) *1. Pre-built Vendor IPs (Provided by AMD Vivado):*

These components are instantiated, configured, and intercon-

nected through the **Vivado IP Integrator (IPI)** GUI or Tcl scripts:

- **Zynq UltraScale+ MPSoC IP:** Manages the ARM Cortex-A53 core, clock generation (e.g., 100 MHz/200 MHz for PL), reset lines, and physical DDR4 memory controller interfaces.
- **AXI Direct Memory Access (AXI DMA) IP:** Handles high-throughput transfers between DDR memory and the FPGA fabric via AXI4-Stream without CPU intervention.
- **AXI SmartConnect / AXI Interconnect:** Connects control registers (AXI4-Lite) and routes memory channels between the PS and PL.
- **Processor System Reset IP:** Coordinates clean reset synchronization between clock domains.
- **(Optional) AXI4-Stream Data FIFO / BRAM Controller:** Standard FIFOs to buffer data bursts before

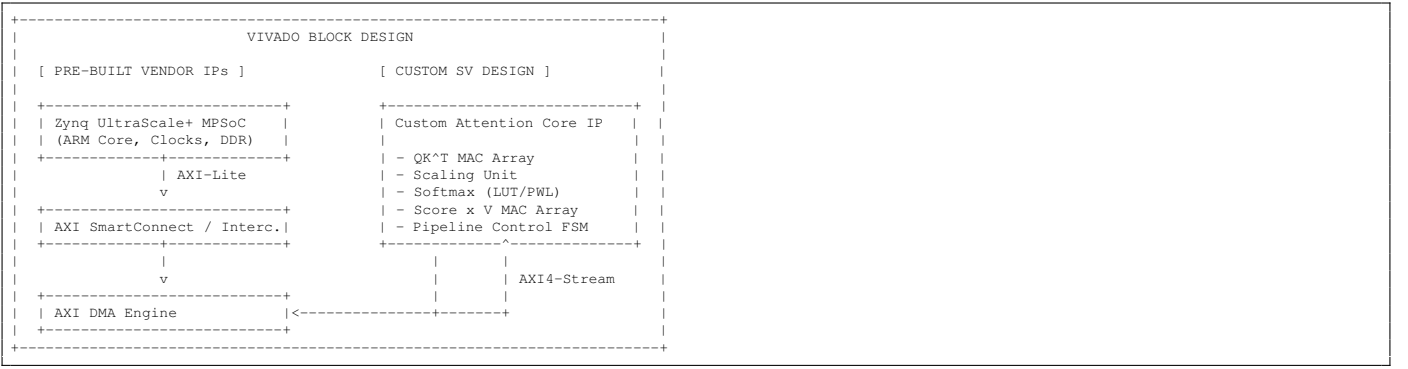


Fig. 4: ASCII architecture or workflow diagram preserved from the source material.

reaching the core.

3) 2. *Custom IP Blocks*: SystemVerilog development focuses strictly on the computational datapath and interface wrappers:

- **The Attention Core Datapath:**
 - QK^T Matrix Multiplication (DSP-based MAC array).
 - Attention Score Scaling (Bit-shift / Fixed-point division).
 - Approximated Softmax Engine (LUT / Piecewise Linear).
 - Score $\times V$ Aggregation Engine.
- **AXI Interface Wrappers (Glue Logic):**
- **AXI4-Stream Interface (S_AXIS & M_AXIS):** Handles handshake protocol (TVALID, TREADY, TDATA, TLAST) to consume raw Q, K, V streams and emit output vectors.
- **AXI4-Lite Slave Interface (S_AXI):** Provides memory-mapped control registers for the ARM PS to set matrix dimensions (sequence length N , head dimension d_k), trigger execution (START), and read execution status (DONE, cycle counters).

4) *Summary Checklist*:

VII. HANDSHAKE PROTOCOLS

In digital hardware design—particularly in System-on-Chip (SoC) and FPGA environments—the **handshake protocol** is the fundamental mechanism used to reliably transfer data between two hardware modules (a Producer and a Consumer) that may be operating at different speeds, processing rates, or even different clock frequencies.

The most ubiquitous and heavily standardized handshake protocol in modern hardware is the **Synchronous VALID/READY Protocol**, which forms the absolute foundation of the **AMBA AXI** (Advanced eXtensible Interface) family, including AXI4-Lite, AXI4-Full, and AXI4-Stream.

The following subsections define handshake operation, critical system-safety rules, timing cases, backpressure, and clock-domain crossing.

1) 1. *The Synchronous VALID/READY Protocol (AXI Standard)*: At its core, the AXI-style handshake relies on just two control signals flowing in opposite directions:

- **VALID (Master $\rightarrow$ Slave)**: Asserted by the sender (Producer) to indicate that the data payload currently on the bus is valid and ready to be read.
- **READY (Slave $\rightarrow$ Master)**: Asserted by the receiver (Consumer) to indicate that it has the internal buffer space or processing capability to accept new data on the current clock cycle.

a) *The Golden Rule of Transfer*: A successful data transaction **only occurs when both VALID and READY are high (logic 1) on the same rising edge of the clock**.

$$\text{transfer} = (\text{VALID} = 1) \wedge (\text{READY} = 1) \quad \text{at } \uparrow \text{clk.} \quad (3)$$

2) 2. *The Three Critical Handshake Rules (Deadlock Prevention)*: To prevent the entire SoC from freezing (deadlock) or losing data, ARM explicitly defines strict rules for the AXI VALID/READY handshake. Violating these in the SystemVerilog code will cause unpredictable hardware failures.

3) *Rule 1: VALID must NEVER depend on READY for assertion.*:

- The Master cannot wait for the Slave to indicate readiness before asserting VALID. If the Master waits for READY while the Slave waits for VALID, a **combinatorial deadlock** is created in which both interfaces wait indefinitely.
- *Correct design*: The Master asserts VALID as soon as it has data, regardless of the state of READY.

4) *Rule 2: Once VALID is asserted, it cannot be de-asserted until the transfer occurs.*:

- If the Master raises VALID, but the Slave's READY is low, the Master **must hold VALID high** and keep the data payload perfectly stable on the bus.
- A request cannot be withdrawn after VALID is asserted. VALID drops only after the clock edge on which READY is also high.

5) *Rule 3: READY MAY depend on VALID, or it may not.*:

- A Slave is allowed to wait for VALID to go high before calculating if it is READY (though this adds latency).
- Alternatively, a high-performance Slave can assert READY proactively (default to 1) before VALID even arrives,

TABLE IV: SoC component sources and project responsibilities.

SoC Component	Source	Project Responsibility
ARM CPU & Clocks	Vivado Catalog (Zynq MPSoC)	Select Kria KV260 preset board configurations.
DDR4 Controller	Vivado Catalog (Zynq MPSoC)	Enabled automatically by KV260 board files.
AXI DMA Engine	Vivado Catalog (AXI DMA)	Configure data stream width (e.g., 32-bit / 64-bit).
AXI Bus Interconnect	Vivado Catalog (AXI SmartConnect)	Auto-connected by Vivado connection automation.
Attention Accelerator	SystemVerilog RTL	Write RTL, simulate, and package the design as custom IP through "Tools → Create and Package New IP."

allowing for instant, zero-latency transfers the moment the Master has data.

6) 3. *Timing Scenarios*: Depending on which module is faster, three scenarios emerge during execution:

- 1) **Zero-Wait Transfer (Fast Receiver)**: The Slave asserts $READY = 1$ continuously. The Master drives data and asserts $VALID = 1$. The transfer happens instantly on the very next clock edge.
- 2) **Master Stalled by Slave (Backpressure)**: The Master asserts $VALID = 1$. The Slave is busy processing previous data, so $READY = 0$. The Master is forced to wait (stalled pipeline) until the Slave finishes and raises $READY = 1$.
- 3) **Slave Starved by Master**: The Slave is waiting with $READY = 1$, but the Master is calculating data and has $VALID = 0$. No transfer occurs.

7) 4. *Pipelining and Backpressure (The "Skid Buffer" Problem)*: Deep FPGA pipelines such as the hardware softmax engine require pipeline registers to meet timing targets such as $F_{max} > 200$ MHz.

However, pipelining a $VALID/READY$ bus requires explicit flow control. Inserting only a standard D flip-flop on the data path is insufficient: when the downstream module deasserts $READY$, data already traveling through the pipeline can reach the stalled boundary and be overwritten or lost.

To solve this, hardware engineers use a **Register Slice (or Skid Buffer)**. A Skid Buffer decouples the timing between a Master and Slave by utilizing a 2-entry FIFO:

- It passes data directly when there is no backpressure.
- If the downstream Slave drops $READY = 0$ suddenly, the Skid Buffer has a spare register (the "skid" slot) to catch the data that was already "in flight" on that exact clock cycle without losing it.
- Once the skid slot is full, the Skid Buffer drops its own upstream $READY = 0$ to halt the Master.

8) 5. *Asynchronous Handshakes (Clock Domain Crossing)*: While AXI $VALID/READY$ is *synchronous* (Master and Slave share the exact same clock), hardware often requires moving data between different clock frequencies (e.g., a 100 MHz ARM PS and a 250 MHz FPGA PL). This requires **Clock Domain Crossing (CDC)** handshakes.

In a clock-domain crossing, a $VALID$ signal cannot be sampled directly because it may change while the receiving clock captures it, causing **metastability**. Asynchronous handshakes instead use a Request/Acknowledge (REQ/ACK) system passed through a two-flip-flop synchronizer chain.

There are two main CDC handshake protocols:

a) A. 4-Phase Handshake (Return-to-Zero / RTZ):

- 1) **Master** asserts $REQ = 1$ (I have data).
 - 2) **Slave** sees REQ , captures data, and asserts $ACK = 1$ (I got it).
 - 3) **Master** sees $ACK = 1$ and must drop $REQ = 0$.
 - 4) **Slave** sees $REQ = 0$ and drops $ACK = 0$.
- *Pros*: Extremely safe, easy to debug.
 - *Cons*: Very slow. It takes multiple clock cycles across both domains just to transfer a single word because the signals must "return to zero" before the next transfer.

b) B. 2-Phase Handshake (Non-Return-to-Zero / NRZ):

Instead of waiting for signals to return to 0, data is transferred every time there is a **toggle** (transition from $0 \rightarrow 1$, or $1 \rightarrow 0$).

- 1) **Master** toggles REQ (e.g., from 0 to 1). Data is valid.
 - 2) **Slave** sees the toggle, captures data, and toggles ACK (from 0 to 1) to match.
 - 3) For the next transfer, **Master** toggles REQ again (from 1 down to 0).
- *Pros*: Doubles the throughput compared to 4-phase.
 - *Cons*: Requires slightly more complex XOR edge-detection logic to implement safely.

9) *Application to The ViT Project*: The Vision Transformer implementation on the Kria KV260 uses **AXI4-Stream** with $TVALID$ and $TREADY$. The softmax and MAC-array RTL blocks must respect backpressure. If the receiving DMA engine pauses with $TREADY$ low, the complete matrix-multiplication pipeline must halt safely without dropping or corrupting intermediate attention scores.

10) *Sources*:

- 1) handshake.adoc - jeras/synthesis-primitives - GitHub
- 2) Overview of AXI4 bus operations - Welcome to Real Digital
- 3) How the AXI-style ready/valid handshake works - VHDLwhiz
- 4) How the AXI-style ready/valid handshake works - YouTube
- 5) Valid-Ready Protocol | Home

VIII. SOC TRAINING PATH FOR RTL DESIGNERS

Transitioning from pure RTL/digital design to SoC hardware acceleration is primarily a change in design abstraction: **the circuit is no longer treated in isolation as pins, clocks, and resets, but as an addressable node on an interconnected system bus.**

Existing knowledge of SystemVerilog, state machines, and datapath pipelining provides the required digital-design foundation. The remaining knowledge gap concerns **SoC bus protocols (AMBA AXI), Direct Memory Access (DMA), and HW/SW interface abstractions.**

1) *The 4-Stage Hands-On Training Path:* The most effective way to train is through progressive "mini-accelerator" milestones in Vivado before building the full Vision Transformer datapath.

```

1 Stage 1: AXI Protocol Fluency (SystemVerilog Level)
2 +-- Understand AXI4-Lite & AXI4-Stream handshakes ->
   Simulate custom protocol wrappers.
3
4 Stage 2: Memory-Mapped Control (PS-PL Register Access)
5 +-- Design an AXI-Lite Slave -> Write/read registers from
   ARM CPU (Python/C).
6
7 Stage 3: Streaming & DMA Transfers (High-Throughput Data
   Movement)
8 +-- Connect AXI DMA IP to an AXI-Stream FIFO -> Transfer
   arrays from DDR to PL and back.
9
10 Stage 4: Stream Coprocessor Integration (Mini-Accelerator
   Spike)
11 +-- Build a 1D Vector Dot-Product IP -> Stream arrays via
   DMA -> Read result -> Bridge to ViT.
```

Listing 3: Source code, configuration, command sequence, or directory structure transcribed verbatim.

2) *IMaster the AMBA AXI Protocol Family:* Learn how modern SoCs talk internally

Modern Xilinx/AMD SoCs use the AMBA AXI4 standard for almost all communication. Focus on these two flavors:

- **AXI4-Lite (Control Plane):** A simple, low-throughput memory-mapped protocol. Used for reading/writing configuration registers (e.g., Matrix dimensions, Start bit, Status registers).
- **AXI4-Stream (Data Plane):** An address-less, point-to-point streaming protocol with no burst limits. Used for pumping high-dimensional tensors (Q, K, V matrices) directly into the compute pipeline.
- **Action Item:** Read the *ARM AMBA AXI and ACE Protocol Specification* (specifically the sections on VALID/READY handshakes and channel architecture) or Xilinx UG1037 (*A Guide to AXI4-Interconnect*).

3) *2Build and Control an AXI4-Lite Slave Register Map:* Connect the first SystemVerilog module to the CPU

Learn how software translates memory writes (e.g., writing to address 0xA000_0000) into hardware control signals.

- **How to do it:** In Vivado, use **Tools** → **Create and Package New IP** → **Create AXI4 Peripheral**. Vivado will generate a template SystemVerilog AXI4-Lite slave with 4 configuration registers (slv_reg0 to slv_reg3).
- **Hands-on Mini-Project:** Modify the template so that slv_reg0 is an input operand A , slv_reg1 is operand B , and slv_reg2 computes $A \times B$.
- **Integration:** Add the Zynq UltraScale+ MPSoC block, instantiate the IP, generate the bitstream, and write a simple script in Python (using PYNQ) or C to write numbers to the registers and read back the product.

4) *3Understand DMA and Memory Buffers:* Master off-chip to on-chip tensor movement

CPU's are too slow to feed millions of tensor elements over registers. DMA allows hardware to read/write directly from DDR4 memory without CPU overhead.

- **Core Concepts:** Learn how the **AXI DMA IP** converts memory-mapped DDR4 data (AXI4-Full) into a stream of raw bytes (AXI4-Stream) via MM2S (Memory-Mapped to Stream), and vice versa via S2MM (Stream to Memory-Mapped).
- **Software Side (CMA & Buffer Allocation):** Learn why normal memory allocation (malloc) does not work with hardware DMA. In Linux/PYNQ, hardware requires **Contiguous Memory Allocation (CMA)** so physical addresses remain sequential.
- **Hands-on Mini-Project:** Build a loopback pipeline in Vivado: Zynq PS → AXI DMA → AXI4-Stream Data FIFO → AXI DMA → Zynq PS. Send an array of 1,000 integers from Python/C and verify the received array matches.

5) *4Build a Stream-Based Vector Arithmetic Coprocessor:* The final bridge to the ViT capstone

Combine AXI-Lite control and AXI-Stream data to create a miniature accelerator that mirrors the exact structure of the ViT Attention core.

- **Module Design:** Create an IP that takes two vectors a and b via AXI-Stream, computes the dot product $\sum_i a_i b_i$ using internal DSP48E2 MAC logic, and streams the scalar result back out.
- **Control Flow:** Use AXI-Lite to set the vector length N , trigger a START pulse, and monitor an IDLE / DONE flag.
- **Significance:** Completion of this stage reduces development of the QK^T attention core to scaling the MAC datapath and adding the softmax stage.

6) *Key Concepts Cheatsheet:*

7) *Recommended Resources to Study:*

- 1) **PYNQ Documentation & Tutorials (pynq.io):** Focus on the *Overlay*, *MMIO (Memory Mapped IO)*, and *AXI DMA* tutorials. This is the fastest way to run the RTL from the ARM core on the Kria KV260.
- 2) **AMD/Xilinx UG1037: Vivado Design Suite AXI Reference Guide** (Chapters 1–3 explain the bus architectures and channel handshakes clearly).
- 3) **PG021: AXI DMA LogiCORE IP Product Guide** (Read the Direct Register / Simple DMA Mode section).

IX. HARDWARE DESCRIPTION LANGUAGE SELECTION

For the capstone's AXI interfaces and complex tensor mathematics in Vivado, **SystemVerilog is the preferred hardware-description language.**

Standard Verilog (Verilog-2001) is a legacy subset of SystemVerilog. Verilog was merged into the IEEE 1800 SystemVerilog standard, whose additional constructs improve both RTL design and verification. AMD Vivado provides

TABLE V: Technical comparison: SoC Concept; Pure RTL Equivalent; Why It Matters for ViT Accelerator.

SoC Concept	Pure RTL Equivalent	Why It Matters for ViT Accelerator
Memory-Mapped (MM)	Register file with address decoder	Used for scalar control (Sequence length N , Head dimension d_k , Start/Reset).
Streaming (AXIS)	FIFO interface (data_in, push, pop, full, empty)	Used for moving high-volume Q, K, V and attention score matrices.
AXI DMA	Automatic memory reader/writer	Reads activation tensors from DDR4 RAM and pushes them directly into the RTL.
Contiguous Buffer (CMA)	Fixed continuous physical RAM block	FPGA DMA cannot follow virtual memory page tables; arrays must be contiguous in physical RAM.
PYNQ Framework	Software test driver	Supports bitstream loading, DMA-buffer allocation, and end-to-end inference in Python.

synthesis and simulation support for these SystemVerilog capabilities.

The selection is supported by the following language and verification capabilities.

1) *1. interface Constructs for AXI Buses:* The design relies heavily on AXI4-Stream to move the Q , K , and V matrices. In standard Verilog, connecting an AXI bus requires explicitly declaring and wiring five to ten separate signals (e.g., tvalid, tready, tdata, tlast, and tkeep) for every module through which it passes. SystemVerilog introduces the interface construct, allowing the AXI bus to be declared once and passed as a single bundle through slave and master modports. This reduces repetition, improves readability, and prevents missed connections.

2) *2. Multi-dimensional Arrays and struct:* The project involves manipulating chunks of matrices and vectors representing attention scores. SystemVerilog supports custom data types (typedef) and multidimensional arrays. A 16×8 -bit vector can therefore be packed into an array and passed natively into the MAC array, whereas standard Verilog requires a flattened bit vector with manually indexed slices.

3) *3. The logic Data Type:* In standard Verilog, engineers constantly trip over the rules of when to declare a signal as a wire (for continuous assignments) versus a reg (for procedural blocks). SystemVerilog solves this by introducing the logic data type. The compiler automatically infers whether it should be synthesized as a wire or a flip-flop register based on context, eliminating one of the most frustrating aspects of RTL design.

4) *4. Hardware Intent (always_comb and always_ff):* Verilog relies on generic always @(*) and always @(posedge clk) blocks. An incomplete sensitivity specification or conditional assignment can silently infer an unintended latch and compromise timing. SystemVerilog introduces always_comb for combinational logic and always_ff for flip-flops; tools can consequently diagnose incompatible hardware intent earlier in the flow.

5) *5. Advanced Testbenches (Verification):* Because softmax is approximated using a lookup table or piecewise-linear arithmetic, verification against the Python/PyTorch numerical reference is a major evaluation requirement. SystemVerilog provides object-oriented testbench features, constrained-random generation, and assertions that support more capable self-checking environments than standard Verilog.

Recommendation: New FPGA source modules should use the .sv extension and employ SystemVerilog’s modern design

and verification features rather than beginning a new codebase in Verilog-2001.

6) *Sources:*

- 1) Should I bother learning Systemverilog for RTL design or just stick
- 2) SystemVerilog vs Verilog in RTL Design
- 3) System verilog - Adaptive Support - AMD
- 4) Verilog vs SystemVerilog: What’s the Difference... - ChipXpert

X. VIVADO VERSION AND RUNTIME WORKFLOW

A newer Vivado release is required for the KV260 workflow (at least Vivado 2022.1, with 2023.2 identified as the project baseline), whereas Vitis HLS is not required for a SystemVerilog implementation.

Vivado 2018 predates the target platform and cannot provide the required native board support. The modern workflow also replaces several steps associated with the earlier SDK-based process.

1) *1. Vivado 2018 Does Not Support the Kria KV260:* The AMD/Xilinx Kria KV260 Vision AI Starter Kit was officially launched in 2021. Because it did not exist in 2018, Vivado 2018 physically does not contain the board files, the K26 System-on-Module (SOM) presets, or the correct Zynq UltraScale+ hardware definitions required to compile a bitstream for it. The earliest version that supports the KV260 natively is Vivado 2021.1.

2) *2. SDK Replacement and Vitis HLS Scope:* In Vivado 2019.2, Xilinx officially retired the **Xilinx SDK** and replaced it with the **Vitis Unified Software Platform**.

However, many engineers confuse **Vitis** with **Vitis HLS**.

- **Vitis HLS (High-Level Synthesis):** Converts C/C++ into hardware. It is not required because the attention core is implemented directly in SystemVerilog.
- **Vitis Core / Vitis IDE:** This environment is the modern replacement for Xilinx SDK and is used for writing bare-metal C/C++ drivers.

3) *PYNQ-Based Alternative to Vitis/SDK:*

4) *3. The Modern Flow: Vivado → PYNQ (Python):* In 2018, the standard workflow was:

- 1) Build hardware in Vivado.
- 2) Export to Xilinx SDK.
- 3) Write complex, low-level C code to configure the DMA and move data.

For the Kria KV260, the modern workflow substantially reduces host-software development effort:

- 1) **Vivado (2022.1+)**: RTL design, block design, and bitstream generation remain in Vivado.
- 2) **Export Hardware**: Instead of exporting to SDK, the flow exports a .bit bitstream and .hwh hardware-handoff file.
- 3) **PYNQ (Ubuntu)**: The KV260 runs an official Ubuntu Linux image with the PYNQ framework. The .bit file is transferred to the board, after which a Jupyter Notebook or Python application controls the hardware.
- 5) *How PYNQ Replaces Xilinx SDK for the Project*: The ML accelerator requires PyTorch Q , K , and V tensors to be transferred to the FPGA. PYNQ replaces substantially more verbose bare-metal C/SDK control code with a compact Python interface:

```
1 from pynq import Overlay, allocate
2 import numpy as np
3
4 # 1. Load the bitstream generated in Vivado
5 overlay = Overlay("attention_core.bit")
6 dma = overlay.axi_dma_0
7
8 # 2. Allocate contiguous memory for the DMA
9 input_buffer = allocate(shape=(196, 64), dtype=np.int8)
10 output_buffer = allocate(shape=(196, 64), dtype=np.int8)
11
12 # 3. Copy the PyTorch data into the buffer
13 np.copyto(input_buffer, my_pytorch_qkv_tensor)
14
15 # 4. Trigger the DMA transfer to the FPGA and wait for the
    result
16 dma.sendchannel.transfer(input_buffer)
17 dma.recvchannel.transfer(output_buffer)
18 dma.sendchannel.wait()
19 dma.recvchannel.wait()
```

Listing 4: Source code, configuration, command sequence, or directory structure transcribed verbatim.

6) Summary Recommendation:

- **Download Vivado ML Standard Edition 2023.2.** (It is free and highly stable).
- **Ignore Vitis HLS.** Stick to SystemVerilog for the hardware.
- **Ignore Vitis IDE/SDK.** Install the official Ubuntu 22.04 LTS image on the Kria KV260's SD card, install PYNQ, and use Python to drive the AXI DMA and test the Vision Transformer.

7) Sources:

- 1) Getting Started with the Kria KV260 in Vivado 2021.1 - Hackster.io

XI. DEVELOPMENT ENVIRONMENT AND PHYSICAL PLATFORM

Development from initial setup through physical demonstration on the AMD Kria KV260 is divided into **three distinct tiers**:

- 1) **Host PC (Laptop / Desktop)**: Used to write SystemVerilog, simulate hardware, design block diagrams, synthesize bitstreams, and simulate quantization.

- 2) **Cloud / High-Compute (Google Colab / Kaggle)**: Used to download datasets and train or fine-tune the lightweight Vision Transformer baseline on GPUs.
- 3) **Target Edge Hardware (Kria KV260 Board)**: The physical embedded device running Linux, hosting the PYNQ framework, and executing the custom FPGA bitstream.

A. Tier 1: Host Development Workstation

This is the main development machine. AMD Vivado is resource-heavy, so ensuring the computer meets the hardware baseline is critical.

1) 1. Hardware Requirements:

- **OS**: Windows 10/11 (64-bit) or Ubuntu Linux 22.04 LTS. *Linux commonly provides faster Vivado build execution, while Windows remains a supported host environment.*
- **RAM: Minimum 16 GB** (32 GB strongly recommended). Insufficient memory can cause Vivado synthesis for Zynq UltraScale+ devices to terminate or become unresponsive.
- **Disk Space: Minimum 80–100 GB free SSD storage** for Vivado installation and project cache files.

2) 2. FPGA & Hardware Design Software:

a) A. **AMD Vivado ML Edition (Version 2022.2 or 2023.2)**:

- **What it is**: The official IDE from AMD/Xilinx used to synthesize SystemVerilog code, run timing simulations, place-and-route circuits, and generate bitstreams (.bit files).
- **License: Vivado ML Standard is 100% Free** (no paid license needed). It includes full support for the Zynq UltraScale+ MPSoC (K26 SOM on the KV260).
- **Components to select during installation**:
- Select **Vivado**; Vitis may be omitted to save approximately 40 GB of disk space.
- In the device tree, check **Zynq UltraScale+ MPSoC**.
- **Mandatory Board Files**:
- Inside Vivado, go to Tools -> Vivado Store -> Boards and install the **Kria KV260 Vision AI Starter Kit** board definition files. This automatically configures DDR4 timings, pin constraints, and clock frequencies.

b) B. Vivado Simulator (xsim):

- **What it is**: The built-in simulator inside Vivado.
- **Purpose**: Runs the SystemVerilog testbenches, measures latency clock cycles, and displays waveforms to verify the MAC array and Softmax math before synthesis.

3) 3. **Code Editor & Extensions: Visual Studio Code (VS Code)** is recommended for all daily coding (SystemVerilog, Python, Markdown documentation).

4) 4. **Host Machine Python Environment**: Manage the local Python packages using **Anaconda**, **Miniconda**, or native venv.

- **Python Version**: 3.10 or 3.11
- **Core Libraries to Install (pip install ...)**:
- torch torchvision: For loading the ViT model, running inference, and extracting intermediate tensors.

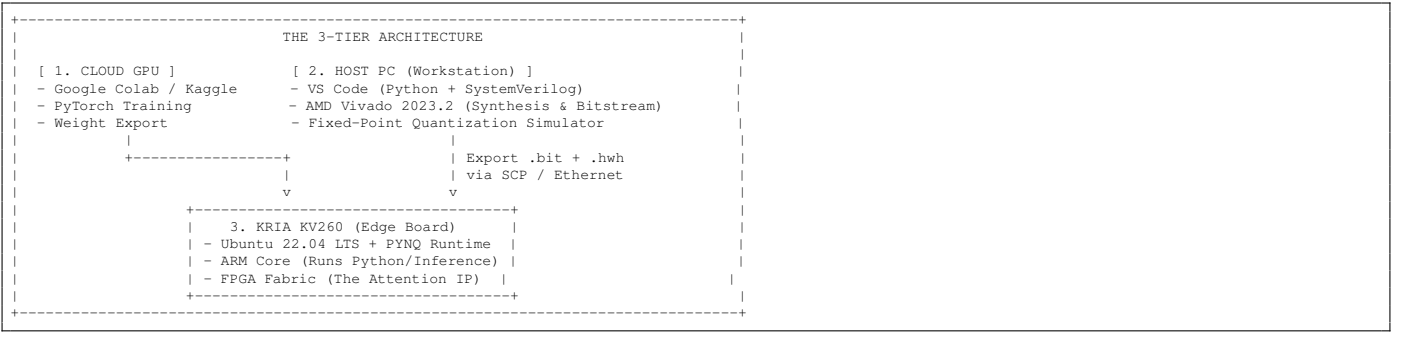


Fig. 5: ASCII architecture or workflow diagram preserved from the source material.

TABLE VI: Technical comparison: Extension Name; Purpose.

Extension Name	Purpose
Verilog-HDL/SystemVerilog (by <i>mshr-h</i>)	Syntax highlighting, code navigation, and linter support for SystemVerilog.
SystemVerilog - Language Support (by <i>EFCoders</i>)	Auto-completion for complex types, interfaces, and structs.
Python (by <i>Microsoft</i>)	Linting, debugging, and running PyTorch simulation scripts.
WaveTrace / Wavetern	Allows viewing .vcd waveform files directly inside VS Code without launching the full Vivado GUI.
Remote - SSH (by <i>Microsoft</i>)	Supports direct editing and execution on the Kria KV260 from the host workstation over Wi-Fi or Ethernet.

- **numpy**: For fixed-point bit manipulation and array formatting.
- **matplotlib seaborn**: For plotting error distributions between floating-point Softmax vs. hardware Softmax.
- **scipy**: For mathematical verification.

B. Tier 2: Cloud / Model Training Environment

An expensive local GPU is not required to train the Vision Transformer baseline.

- **Platform: Google Colab or Kaggle Notebooks** (Free Tier).
- **Role in Project:**
 - 1) Download the target dataset (e.g., UCF101 activity recognition, or custom occupancy/traffic images).
 - 2) Fine-tune a lightweight Vision Transformer (e.g., MobileViT-XXS or TinyViT).
 - 3) Export the trained model weights (.pth / .pt files) back to the local machine.

C. Tier 3: Edge Embedded Environment (Kria KV260)

The AMD Kria KV260 runs a full Linux distribution on its ARM quad-core Cortex-A53 processor.

1) 1. Operating System (MicroSD Card):

- **OS Image: Canonical Ubuntu 22.04 LTS for Kria** (downloadable directly from the AMD/Xilinx Kria App Store / Ubuntu certified portal).
- **Flashing Tool: BalenaEtcher or Raspberry Pi Imager** (to flash the OS image onto a MicroSD card).

2) 2. Runtime Framework: PYNQ:

- **What it is:** Python on Zynq. A framework developed by AMD that allows Python running on the ARM CPU to directly control FPGA bitstreams and DMA transfers.

- **Why it is essential:** It avoids the need to create custom bare-metal C drivers or Linux kernel modules for basic hardware control.

3) 3. **Board-Side Python Libraries:** Installed directly on the Kria Ubuntu environment via terminal:

- **pynq**: Hardware overlay and DMA control.
- **opencv-python-headless**: For camera image capture and resizing.
- **numpy**: For handling DMA input/output memory buffers.

D. Tier 4: Physical Hardware & Bench Tools

E. Complete Software Checklist

To get started today, set up the following tools:

```

1 [ ] 1. Install AMD Vivado ML Standard (Version 2022.2 or
2 [ ] 2. Install Kria KV260 Board Files in Vivado Store
3 [ ] 3. Install VS Code with SystemVerilog and Python
4 [ ] 4. Install Python (Miniconda) with PyTorch, Torchvision
5 [ ] 5. Install BalenaEtcher and download Ubuntu 22.04 LTS
6 [ ] 6. Install PuTTY (or the preferred terminal emulator)

```

Listing 5: Source code, configuration, command sequence, or directory structure transcribed verbatim.

XII. HOST OPERATING-SYSTEM STRATEGY

VirtualBox is not required for the Kria KV260 PYNQ workflow. WSL2 is also optional and is relevant primarily when a Linux environment is preferred for Python development.

Many tutorials specify a Linux VM because older Xilinx workflows required the Linux-only **PetaLinux** tool to build the operating system. The selected **PYNQ + Ubuntu workflow** does not require a custom PetaLinux build.

The host and target operating-system responsibilities are separated as follows.

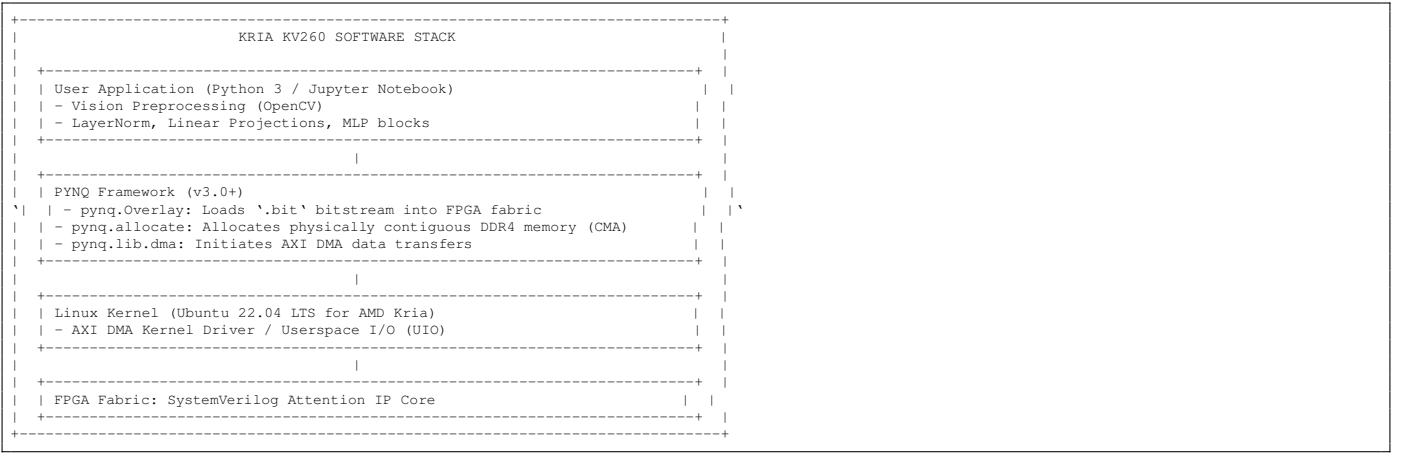


Fig. 6: ASCII architecture or workflow diagram preserved from the source material.

TABLE VII: Technical comparison: Hardware Item; Purpose / Specification.

Hardware Item	Purpose / Specification
AMD Kria KV260 Starter Kit	The main evaluation board containing the K26 SOM.
MicroSD Card	32 GB or 64 GB (Class 10 / UHS-I / V30 speed) to store Ubuntu and datasets.
Power Supply	12V DC, 3A (36W) barrel jack (Center-Positive, 5.5mm outer / 2.1mm inner).
Micro-USB to USB-A Cable	Connects from Kria's JTAG/UART port to the PC for serial console debugging.
Ethernet Cable	Connects the KV260 to the home router or laptop to enable SSH and Jupyter Lab.
Serial Terminal Software (PC)	PuTTY (Windows) or Minicom / TeraTerm (to view the Linux boot log via COM port).
(Optional) USB Webcam	Any standard UVC USB webcam (for live video inference tests).

1) *1. The Host PC (Windows 10/11)*: All hardware-design tasks can be performed natively on Windows.

- **Vivado**: The **Windows Native Installer** for Vivado ML Standard 2023.2 generates the `.bit` bitstream files required by the KV260 workflow.
- **Python (PyTorch)**: Python can be installed directly on Windows through Anaconda for fixed-point simulation and test-vector generation.

2) *2. The Target Board (Kria KV260)*: This is where Ubuntu actually runs.

- Ubuntu is not booted on the host PC for deployment. The official AMD Ubuntu 22.04 image is downloaded, flashed onto a MicroSD card using BalenaEtcher on Windows, and inserted into the KV260 board.
- The **KV260's ARM processor** boots Ubuntu. The board is then accessed from the Windows PC through a web browser, Jupyter Notebook, or SSH using PuTTY or VS Code Remote-SSH.

3) *Optional Linux Host Environment: WSL2*: When a Linux environment is preferred for PyTorch training scripts or Git commands, **WSL2 (Windows Subsystem for Linux)** is preferred to VirtualBox.

4) *Limitations of VirtualBox for FPGA Development*:

- 1) **Memory contention**: Vivado requires at least 16 GB of RAM to synthesize Zynq UltraScale+ designs reliably. Allocating this amount to a VirtualBox VM can starve the Windows host, whereas WSL2 manages memory dynamically.

- 2) **USB passthrough**: JTAG debugging and serial-console access require reliable micro-USB connectivity. Native Windows access avoids the additional VirtualBox USB-passthrough layer.

5) *Summary*: Stick to **Native Windows** for Vivado. Use **Native Windows** or **WSL2** for the Python PyTorch scripts. Let the **Kria KV260** handle the Ubuntu booting.

6) *Sources*:

- 1) Vivado, Vitis and PetaLinux on Windows Sublayer for Linux (WSL2)

XIII. VS CODE-CENTERED ENGINEERING WORKFLOW

Using **Visual Studio Code (VS Code)** as the primary development environment—while retaining Vivado for synthesis and bitstream generation—provides a unified hub for:

- 1) **Python & PyTorch** (Quantization simulation, golden test vector generation).
- 2) **SystemVerilog RTL** (Writing, linting, syntax checking, and waveform viewing).
- 3) **Kria KV260 Remote Control** (Writing PYNQ scripts and running live inference on the board via SSH).

The following setup defines the required extensions, local environments, remote access, and repository organization.

A. 1. The VS Code Extension Toolkit

Install these core extensions from the VS Code Marketplace (Ctrl + Shift + X or Cmd + Shift + X):

TABLE VIII: Technical comparison: Category; Extension Name; Extension ID; Purpose.

Category	Extension Name	Extension ID	Purpose
SystemVerilog	Verilog-HDL/SystemVerilog/Bluespec	mshr-h.verilogHDL	Syntax highlighting, code folding, and linter integration.
SystemVerilog	SystemVerilog - Language Support	cirikpre.systemverilog	Auto-completion for interfaces, structs, and modules.
Waveforms	WaveTrace (or $\mathcal{O}(N^2)$)	wavetrace.wavetrace	Open and inspect .vcd simulation waveforms directly in VS Code tabs.
Python	Python & Pylance	ms-python.python	Python linting, IntelliSense, environment switching, and debugging.
Jupyter	Jupyter	ms-toolsai.jupyter	Run interactive data analysis and error-distribution plots inside VS Code.
Edge Hardware	Remote - SSH	ms-vscode-remote.remote-ssh	Connect directly into the Kria KV260 Ubuntu environment.

B. 2. Setting Up the Local Python Environment

VS Code on the host PC is used to develop the PyTorch baseline and fixed-point quantization scripts.

1) Select Python Interpreter:

- Press Ctrl + Shift + P → type Python: Select Interpreter.
- Choose the project's Conda or virtual environment (e.g., Python 3.10.x ('base')).

1) Interactive Development:

- Standard .py files can use # %% cell markers for interactive Jupyter execution within the editor.

C. 3. Configuring SystemVerilog Linting & Formatting

To catch RTL syntax errors and missing module ports *before* opening Vivado, configure a local linter in VS Code.

1) A. Recommended Fast Linter: Verible (by Google):

- Download the latest **Verible** release binary for Windows (or Linux) and extract it to a folder (e.g., C:\tools\verible\).
- Add the bin folder to the system PATH.
- In VS Code, open Settings (Ctrl + ,), search for verilog.linting, and configure it to use Verible.

2) *B. Project .vscode/settings.json Template:* Create a folder named .vscode in the project root and add a settings.json file:

```
1 {
2   "verilog.linting.linter": "verible-verilog-lint",
3   "verilog.formatting.verilog-format.path": "verible-verilog-
4     format",
5   "verilog.languageServer.enabled": true,
6   "systemverilog.includeIndexing": [
7     "rtl/**/*.sv",
8     "tb/**/*.sv"
9   ],
10  "python.analysis.typeCheckingMode": "basic",
11  "files.associations": {
12    "*.sv": "systemverilog",
13    "*.svh": "systemverilog",
14    "*.v": "verilog"
15  }
```

Listing 6: Source code, configuration, command sequence, or directory structure transcribed verbatim.

D. 4. Connecting VS Code Directly to the Kria KV260 (Remote-SSH)

VS Code Remote - SSH provides direct access to folders on the KV260 and avoids editing through a separate terminal or browser-based interface.

1) *1Get the Kria Board's IP Address:* Connect both the PC and KV260 to the same local network

Connect the KV260 to the router/PC via an Ethernet cable. Power on the board and find its IP address (either via the router's client list or running hostname -I on the serial console).

2) *2Configure SSH Host in VS Code:* Store the target board credentials

- Click the blue **Remote Window icon** in the bottom-left corner of VS Code (or press Ctrl + Shift + P → Remote-SSH: Open SSH Configuration File...).
- Add the host entry:

```
1 Host kria-kv260
2 HostName 192.168.1.150 # Replace with the KV260's actual
3   IP
4 User ubuntu
```

Listing 7: Source code, configuration, command sequence, or directory structure transcribed verbatim.

3) *3Open Remote Workspace:* Work directly on the KV260 filesystem

- Click the bottom-left Remote icon → select **Connect to Host...** → choose kria-kv260.
- Enter the password (ubuntu by default).
- Click **File** → **Open Folder** and select /home/ubuntu/pynq_vit_project.
- The resulting remote session provides terminal access, file browsing, and Python execution on the ARM Cortex-A53 CPU.

E. 5. End-to-End Workflow Map

The daily engineering workflow proceeds through the following VS Code-centered stages.

F. 6. Recommended Project Workspace Structure

Open this root folder inside VS Code:

```
1 vit-accelerator/
2 +-- .vscode/
3 |   +-- settings.json # SV linter & Python
4 |   +-- extensions.json # Recommended team
5 +-- python_sim/
6 |   +-- model.py # PyTorch baseline ViT
7 |   +-- quant_attention.py # INT8/INT16 approximation
8 |   +-- export_vectors.py # Generates test vectors
9 +-- rtl/
```

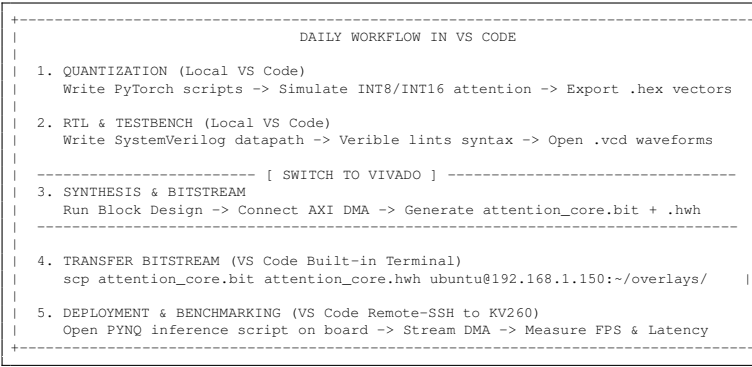


Fig. 7: ASCII architecture or workflow diagram preserved from the source material.

```

10 | +-- pkg_attention.sv      # Custom structs, types,
    | parameters
11 | +-- mac_pe.sv           # Processing element /
    | multiplier
12 | +-- systolic_array.sv  # QK / Score-V compute
    | engine
13 | +-- softmax_lut.sv     # Lookup-table softmax
    | unit
14 | +-- attention_top.sv   # Top module with AXI-
    | Stream interface
15 | +-- tb/
16 | +-- tb_attention_top.sv # Self-checking
    | SystemVerilog testbench
17 | +-- vectors/          # .hex stimulus & golden
    | outputs
18 | +-- scripts/
19 | +-- run_sim.bat        # Batch script to trigger
    | Vivado xsim in background
20 | +-- pynq_host/         # Scripts synced to the
    | KV260 via Remote-SSH
21 | +-- driver.py          # PYNQ DMA controller &
    | overlay loader
22 | +-- benchmark_app.py   # End-to-end latency &
    | accuracy evaluator

```

Listing 8: Source code, configuration, command sequence, or directory structure transcribed verbatim.

XIV. LIGHTWEIGHT VISION TRANSFORMER SCOPE

Defining the model scope for a hardware project establishes explicit mathematical boundaries: matrix dimensions, data types, and the functions assigned to or excluded from the FPGA.

Mapping a standard Vision Transformer (ViT) directly onto a small edge FPGA such as the Kria KV260 can rapidly exhaust memory (BRAM) and multiplier (DSP) resources. The boundaries of a **Lightweight ViT** must therefore be defined explicitly.

The scope is defined through model complexity, dimensional limits, numerical precision, and hardware/software partitioning.

1) *1. Characteristics of a Lightweight ViT:* A standard Vision Transformer such as ViT-Base has **86 million parameters**. Processing a single image generates large matrices with substantial memory-bandwidth requirements.

A lightweight ViT such as **MobileViT-XXS** or **TinyViT-5M** reduces these requirements through the following techniques:

- 1) **Fewer Parameters:** Using between 1 million and 5 million parameters instead of 86 million.

- 2) **Hybrid Designs:** Using standard convolutions (like MobileNet) to process the image early on, and only using Transformers at the very end when the image representation has shrunk to a tiny grid.
- 3) **Fewer Attention Heads:** Standard ViTs look at data from 12 different "perspectives" (heads). Lightweight models only use 1 to 4 heads.

2) *2. The Dimensionality Scope (The Math Boundaries):* SystemVerilog hardware arrays require a fixed maximum size and cannot change physical dimensions dynamically as software arrays do. The capstone scope must therefore establish the following maximum dimensions:

- **Sequence Length ($N \leq 196$):**
 - *Concept:* An image is chopped into a grid of patches (like a puzzle). N is the number of puzzle pieces.
 - *Why it matters:* Attention multiplies a matrix by its transpose (QK^T), creating an $N \times N$ matrix. If $N = 196$ (a 14×14 grid), the hardware must store $196 \times 196 = 38,416$ numbers. If $N = 1024$, the matrix exceeds one million elements and can exceed the FPGA memory budget.
- **Head Dimension ($d_k \leq 64$):**
 - *Concept:* This is how many numbers are used to describe a single puzzle piece.
 - *Why it matters:* In hardware, d_k determines how many multipliers or DSP slices are required in parallel to compute one attention score in a single clock cycle. Values of 32 or 64 provide practical implementation targets for the Kria KV260.
- **Number of Heads ($H \leq 4$):**
 - *Concept:* The number of parallel attention mechanisms running at the same time.
 - *Why it matters:* More heads mean more data streaming back and forth from the DDR RAM. The project should limit this to 1 to 4 heads.

3) *3. The Precision Scope (Data Types):* Floating-point values (e.g., 3.14159) require comparatively large arithmetic circuits. Fitting the ViT onto the FPGA therefore requires constrained numerical data types.

- **Software (Python/ARM):** The model will be trained in Python using **FP32** (32-bit floating point) for high

accuracy.

- **Hardware (FPGA/SystemVerilog):**
- The Q, K, V inputs will be squashed into **INT8** (8-bit integers) using quantization.
- The Multiplier Accumulators (MACs) will temporarily hold **INT32** (32-bit integers) so the math doesn't overflow when adding numbers together.
- The Softmax output will be scaled back down to **INT8** before multiplying with the V matrix.

4) *4. Hardware/Software Partitioning Scope:* The implementation must clearly define which functions run on the ARM processor and which run on the FPGA fabric. Mapping the complete model to the FPGA is outside the defined project scope.

- **Software Scope (Runs on the ARM CPU in Python):**
- Camera capture and image resizing (224×224 pixels).
- Patch extraction (splitting the image).
- Linear Projections (creating the initial Q, K, V weights).
- Layer Normalization (math that is annoying to do in hardware).
- MLP/FFN blocks (standard neural network layers).
- **Hardware Scope (Runs on the custom FPGA circuit):**
- QK^T (Query-Key Matrix Multiplication).
- Scaling by d_k (Done via a simple bit-shift).
- **The Softmax Approximation** (The core challenge of the capstone).
- $\text{Score} \times V$ (Attention Score-Value Matrix Multiplication).

5) *Example Capstone Scope Statement:* The model scope can be expressed through the following specification matrix:

6) *Sources:*

- 1) TinyViT: Fast Pretraining Distillation for Small Vision Transformers

XV. MOBILEViT ARCHITECTURAL RATIONALE

The fundamental difference between a standard Vision Transformer (ViT) and MobileViT is how they handle **spatial relationships** (understanding that a pixel is physically next to another pixel).

Standard ViTs learn these relationships directly through attention, which requires substantial data and computation. MobileViT instead uses Convolutional Neural Networks (CNNs) for early local-feature extraction and reserves Transformer computation for later compressed stages.

1) *Limitations of Standard ViT (ViT-Base):* In a standard ViT, an image is immediately chopped into a grid of patches (e.g., 16×16 pixels). These patches are flattened into 1D vectors and fed directly into the Transformer.

- **Zero Inductive Bias:** A Transformer has no inherent understanding of 2D geometry. It doesn't know that Patch A is located above Patch B until it learns it through millions of training iterations using "Positional Embeddings."
- **Quadratic Scaling:** The attention mechanism compares every patch with every other patch. For an image with N patches, the compute cost scales as $\mathcal{O}(N^2)$.

- **Hardware cost:** Because it relies extensively on attention, a standard ViT stacks 12 large Transformer blocks and reaches **86 million parameters**. This workload can exceed the DDR bandwidth and on-chip BRAM capacity of edge devices.

2) *MobileViT Hybrid Architecture:* MobileViT was designed by Apple specifically for mobile and edge devices. It operates on the philosophy that **CNNs are great at local details, and Transformers are great at the big picture**.

- 1) **Early Processing (CNNs):** Instead of immediately chopping the image into patches, MobileViT runs the raw image through standard MobileNet-style convolutional layers (Inverted Residual blocks). These CNN layers efficiently extract local features (edges, textures) and down-sample the image's resolution without losing information.
- 2) **The MobileViT Block (Unfold & Fold):** Once the image representation is small enough (e.g., a 14×14 feature map), it enters the MobileViT block.
- **Unfold:** The feature map is treated as a grid of "pixels."
- **Global Attention:** A very lightweight Transformer applies Attention across this smaller grid, allowing the network to see the "big picture" (e.g., connecting a dog's ear in the top-left to its tail in the bottom-right).
- **Fold:** The data is packed back into a standard 2D feature map.

3) *Suitability of MobileViT for FPGA Edge Vision:* When mapping a Vision Transformer to a Zynq UltraScale+ device such as the Kria KV260, the principal scaling constraint is the $N \times N$ attention-matrix size.

Offloading the initial convolutions to the ARM Processing System and streaming only the final down-sampled feature maps to the Programmable Logic keeps the sequence length N small. This reduces the storage requirement of the custom QK^T matrix and softmax engine to a range intended to fit within the available FPGA Block RAM.

The Waterfall project plan, module decomposition, and Activity-on-Node (AON) network are defined for **Hardware-Accelerated Transformer Attention for Intelligent Edge Vision (MobileViT on AMD Kria KV260)**.

XVI. PART 1: WATERFALL PROJECT MANAGEMENT PHASES

In a classical Waterfall model for an embedded hardware/software co-design project, each phase must produce clear exit deliverables before the next phase begins to eliminate costly RTL refactoring.

A. Phase 1: Requirements, Learning & Environment Setup

Focus: Toolchain installation, literature review, baseline model selection, and SoC fundamentals.

- **Step 1.1: Environment Initialization**
- Install Vivado ML Standard 2023.2 with KV260 board files.
- Set up VS Code with Verible, Python 3.10, PyTorch, and Remote-SSH.

TABLE IX: Technical comparison: Parameter; Defined Scope Limit; Justification for FPGA.

Parameter	Defined Scope Limit	Justification for FPGA
Target Architecture	MobileViT-XXS / TinyViT-5M	< 5M parameters prevents DDR memory bottlenecking.
Input Resolution	224×224 or 256×256 pixels	Standard edge-vision camera resolution.
Max Sequence Length (N)	196 (from a 14×14 feature map)	Ensures the $N \times N$ attention matrix fits entirely inside on-chip BRAM.
Head Dimension (d_k)	32 or 64	Provides regular mapping to DSP48E2 multiplier arrays on the KV260.
Attention Heads (H)	1 to 4 heads	Keeps AXI-Stream data bandwidth within acceptable limits.
Data Precision	INT8 (Inputs) / INT32 (Accumulators)	Replaces floating-point math, saving ~70% of logic resources.
Softmax Implementation	Lookup Table (LUT) or Piecewise	Eliminates the need for hardware division and exponential blocks.

TABLE X: Technical comparison: Metric; Standard ViT-Base; MobileViT-XXS; Impact on FPGA Design.

Metric	Standard ViT-Base	MobileViT-XXS	Impact on FPGA Design
Parameters	86.0 Million	1.3 Million	Fits entirely in cache; prevents DDR4 bottlenecking.
FLOPs (Compute)	~17.5 Billion	~0.4 Billion	Requires vastly fewer DSP48E2 multiplier arrays.
Inductive Bias	None (Learned)	Convolutional	Reaches high accuracy faster on smaller datasets.
Attention Workload	Heavy (All Layers)	Light (Final Layers)	Only a small number of MobileViT blocks near the end of the pipeline require hardware acceleration.

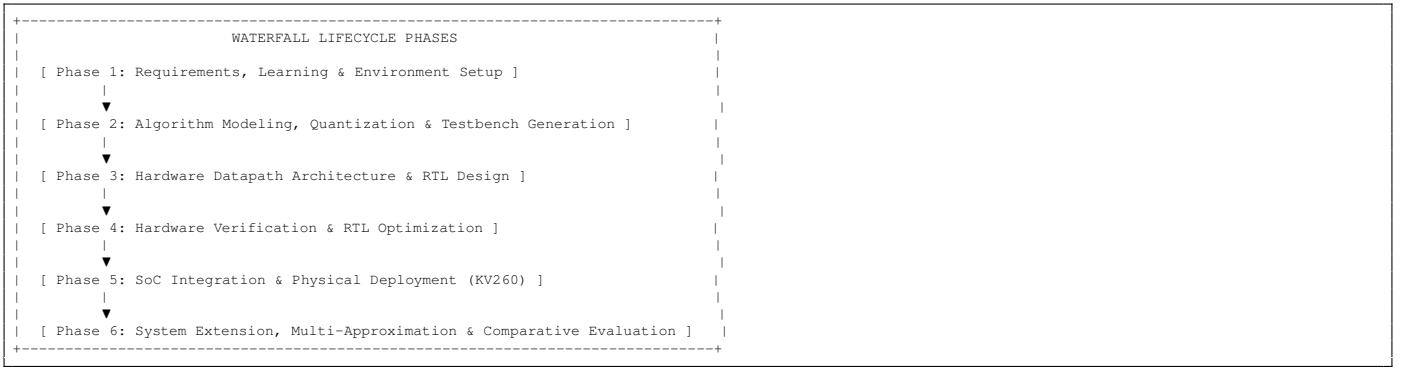


Fig. 8: ASCII architecture or workflow diagram preserved from the source material.

- Flash Kria Ubuntu 22.04 LTS onto MicroSD and configure the PYNQ framework.
- **Step 1.2: Model & Dataset Scoping**
- Select dataset (e.g., CIFAR-100, custom traffic/occupancy dataset, or UCF101 subset).
- Select model backbone: **MobileViT-XXS** (sequence length $N=196$, head dimension $d_k=16$ or 32, heads $H=2$ to 4).
- **Step 1.3: Knowledge Consolidation**
- Study AMBA AXI4-Stream (TVALID/TREADY handshakes) and AXI DMA simple mode.
- Research hardware softmax approximation methods (LUT vs. Piecewise-Linear).
- **Deliverables:** Working PyTorch training script, functional Vivado environment, and verified SSH connection to KV260.

B. Phase 2: Algorithm Modeling, Quantization & Test Vector Generation

Focus: Software reference implementation, fixed-point math simulation, and golden vector export.

- **Step 2.1: Floating-Point Baseline Training**
- Train/fine-tune MobileViT-XXS in PyTorch (FP32) on Google Colab/local GPU.
- Measure baseline accuracy (Top-1 / Top-5 classification accuracy).
- **Step 2.2: Fixed-Point Quantization & Emulation**
- Quantize activations and weights to **INT8** ($Q_{m,n}$ format); accumulators to **INT32**.
- Simulate mathematical approximations in Python:
- Approximation Method A: **LUT-based Softmax** (e^x pre-calculated table).
- Approximation Method B: **Piecewise-Linear (PWL) Softmax** ($2^{x \log_2(e)}$ with bit-shifts).
- Measure accuracy degradation against FP32 baseline.

- **Step 2.3: Golden Test Vector Generation**
- Extract intermediate Q, K, V tensor activations from a specific MobileViT block.
- Write Python exporter script to generate .hex stimuli files for SystemVerilog \$readmemh.
- **Deliverables:** Quantized PyTorch model, approximation error report (MSE vs. FP32), and .hex test vector test suite.

C. Phase 3: Hardware Datapath Architecture & RTL Design

Focus: Custom SystemVerilog IP design for the core attention mechanism.

- **Step 3.1: Micro-Architecture Specification**
- Define bit-widths, pipeline stages, buffer sizing, and state machine transitions.
- **Step 3.2: RTL Core Modules Implementation**
- mac_array.sv: Parameterized Processing Element (PE) array for QK^T and $\text{Score} \times V$ matrix multiplications using DSP48E2.
- scale_unit.sv: Arithmetic bit-shifter for $1/\sqrt{d_k}$ scaling.
- softmax_lut.sv (Method A): Max-subtraction logic, BRAM-based exponential LUT, accumulator, and reciprocal normalizer.
- bram_buffer.sv: Ping-Pong (double-buffered) BRAM controllers to hide memory latency.
- **Step 3.3: AXI Protocol Wrapper**
- axis_slave_adapter.sv: Receives incoming Q, K, V streams with backpressure support.
- axis_master_adapter.sv: Transmits the final attended output vector with TLAST generation.
- axi_lite_regs.sv: Control register map (START, BUSY, DONE, SEQ_LEN, HEAD_DIM).
- **Deliverables:** Complete SystemVerilog source code for the standalone Attention IP Core.

D. Phase 4: Hardware Verification & RTL Optimization

Focus: Cycle-accurate simulation, bug fixing, and timing closure.

- **Step 4.1: Unit & Integration Simulation**
- Write self-checking SystemVerilog testbenches (tb_attention_core.sv).
- Load .hex test vectors via \$readmemh, compare hardware output against Python golden vectors, and assert zero data loss during simulated backpressure (TREADY = 0).
- **Step 4.2: Pipelining & Retiming Optimization**
- Insert pipeline registers across MAC accumulation trees.
- Eliminate combinational loops and long setup paths to target clock frequency $F_{\max} \geq 150\text{--}200$ MHz.
- **Step 4.3: Vivado Synthesis & Resource Budget Check**
- Synthesize the standalone IP in Vivado.
- Verify resource utilization fits comfortably within KV260 limits (LUT < 60%, BRAM < 70%, DSP < 70%).
- **Deliverables:** Passing simulation testbenches, waveform dumps (.vcd), and standalone synthesis timing reports.

E. Phase 5: SoC Integration & Physical Deployment (Phase 1 Capstone Objective)

Focus: Vivado Block Design, bitstream creation, and end-to-end inference on KV260.

- **Step 5.1: Vivado Block Design (IP)**
- Package custom SystemVerilog module as a reusable Vivado IP.
- Instantiate **Zynq UltraScale+ MPSoC, AXI Direct Memory Access (AXI DMA)**, and **AXI SmartConnect**.
- Connect clocks, resets, interrupts, and AXI buses.
- **Step 5.2: Bitstream Generation & Hardware Handoff**
- Run Implementation, generate .bit (bitstream) and .hwh (hardware handoff) files.
- Validate setup/hold slack (WNS > 0 ns, WHS > 0 ns).
- **Step 5.3: PYNQ Driver & End-to-End Application**
- Write Python driver script on KV260:
 - 1) Capture image / load test batch.
 - 2) Run CNN stem + patch extraction on ARM CPU.
 - 3) Offload MobileViT attention blocks to PL via AXI DMA.
 - 4) Receive output, run final classification head on ARM CPU.
- **Step 5.4: Baseline Benchmarking**
- Measure FPS, end-to-end latency, PL execution cycles, and classification accuracy.
- **Deliverables:** Deployed physical demo on Kria KV260, working PYNQ overlay, and Phase 1 milestone report.

F. Phase 6: System Extension, Multi-Approximation & Evaluation (Phase 2 Capstone Objective)

Focus: Second softmax implementation, hardware counters, and comparative trade-off analysis.

- **Step 6.1: Alternative Softmax Engine Design**
- Implement softmax_pwl.sv (Piecewise-Linear / Base-2 Shift Approximation) in SystemVerilog.
- **Step 6.2: Hardware Instrumentation & Telemetry**
- Add 32-bit hardware performance counters to RTL (Total Cycles, Memory Wait/Stall Cycles, Active Compute Cycles).
- **Step 6.3: Parameterization & Flexibility**
- Parameterize RTL to support variable sequence lengths ($N \in \{64, 196\}$) and head counts ($H \in \{1, 2, 4\}$) without recompiling the bitstream.
- **Step 6.4: Comprehensive Comparative Evaluation**
- Benchmark LUT Softmax vs. PWL Softmax under identical workloads across 6 quantitative metrics:
 - 1) End-to-end latency (ms) & Throughput (FPS / GOPS).
 - 2) Numerical Error (MSE / Cosine Similarity vs. FP32).
 - 3) Final Task Accuracy (Top-1 / Top-5 %).
 - 4) FPGA Resource Utilization (LUT, FF, DSP48E2, BRAM18K).
 - 5) DMA Memory Traffic & Stall Cycles.
 - 6) Dynamic Power Consumption (via Vivado Power Analyzer / KV260 PMBus).
- **Deliverables:** Final Capstone Thesis, comparison charts, demonstration video, and reproducible Git repository.

XVII. PART 2: MODULE DECOMPOSITION

XVIII. PART 3: ACTIVITY-ON-NODE (AON) NETWORK & DEPENDENCY TABLE

- 1) *Dependency Table*:
- 2) *AON Diagram (Activity-on-Node Graph)*:

XIX. PART 4: APPROXIMATION METHODS & OPTIMIZATION TECHNIQUES MATRIX

Use this matrix to guide the experiments and evaluation in Phase 2 and Phase 6:

XX. FIXED-POINT ARITHMETIC

In digital hardware design—especially when building neural network accelerators on FPGAs—using floating-point arithmetic (like IEEE 754) is highly inefficient. Floating-point units consume massive amounts of logic to dynamically align exponents and normalize mantissas.

Fixed-point arithmetic solves this by locking the "binary point" in place. To the hardware (like the DSP48 slices on the Kria KV260), **fixed-point math is just standard integer math**. The fractional meaning exists purely in the mind of the designer and how the bits are shifted.

The analysis covers fixed-point representation, conversion, arithmetic, rescaling, overflow control, and hardware implementation.

A. 1. The Q -Format Representation

To express where the imaginary binary point lies, engineers use the **Q -Format Notation**.

While notation conventions vary slightly across the industry (e.g., Texas Instruments vs. ARM), a standard and unambiguous definition is $Q_{m,n}$:

- m = Number of integer bits (including the 2's complement sign bit).
- n = Number of fractional bits.
- N = Total word length ($N = m + n$).

1) *Understanding the Weights*: Let's analyze an 8-bit signed number in $Q_{4,4}$ format ($m = 4, n = 4$). The bits are weighted as follows:

- **Resolution (Step Size)**: The smallest measurable change is the LSB weight: $2^{-n} = 2^{-4} = 0.0625$.
- **Dynamic Range**: The range is $[-2^{m-1}, 2^{m-1} - 2^{-n}]$. For $Q_{4,4}$, this is $[-8.0, +7.9375]$.

B. 2. Converting Between Float and Fixed-Point

When generating golden test vectors in Python/PyTorch, floating-point tensors must be converted into fixed-point integers before transmission to the FPGA.

1) *Formula: Float $\rightarrow$ Fixed*:

$$X_{\text{fixed}} = \text{round}(X_{\text{float}} 2^n). \quad (4)$$

2) *Formula: Fixed $\rightarrow$ Float*:

$$X_{\text{float}} \approx X_{\text{fixed}} 2^{-n}. \quad (5)$$

3) *Example: Encoding Values into $Q_{4,4}$* :

- **Target Float**: 3.625
- **Scale**: $3.625 \times 2^4 = 3.625 \times 16 = 58$.
- **Integer 58 in 8-bit binary**: 0011_1010.
- **Target Float**: -1.5.
- **Scale**: $-1.5 \times 16 = -24$.
- **Integer -24 in 8-bit two's-complement**: 1110_1000.

C. 3. Addition and Subtraction: The Alignment Rule

In fixed-point arithmetic, **numbers can only be added or subtracted directly when their binary points are aligned**. Adding a $Q_{4,4}$ number to a $Q_{2,6}$ number therefore requires one operand to be shifted into a matching representation.

Furthermore, adding two N -bit numbers produces an $(N+1)$ -bit result to safely hold the carry bit.

1) *Example: $3.625 + (-1.5)$ in $Q_{4,4}$* : Both numbers are $Q_{4,4}$, so their points are already aligned.

```

1 0011.1010 (3.625)
2 + 1110.1000 (-1.5)
3 -----
4 10010.0010 (1.750) <-- Notice the result expanded to 9
                        bits (Q5.4)

```

Listing 9: Source code, configuration, command sequence, or directory structure transcribed verbatim.

If we evaluate 0010_0010 in $Q_{4,4}$, then $2^1 + 2^{-3} = 2 + 0.125 = 2.125$. Thus, $3.625 - 1.5 = 2.125$. The math is perfectly correct, and the hardware simply did a standard integer addition.

D. 4. Multiplication and Rescaling (The Tricky Part)

Multiplication in fixed-point causes the bit-width to double. The fundamental rule of fixed-point multiplication is: **multiplying Q_{m_1, n_1} by Q_{m_2, n_2} yields a product in $Q_{(m_1+m_2), (n_1+n_2)}$** .

1) *Example: $3.25 \times (-1.5)$* : Let's multiply 3.25 (0011.0100 in $Q_{4,4}$) by -1.5 (1110.1000 in $Q_{4,4}$).

- 1) **Hardware Integer Multiplication**: $52 \times (-24) = -1248$.
- 2) **Binary Result (16 bits)**: -1248 in 16-bit two's-complement is 1111_1011_0010_0000.
- 3) **Applying the Q -Rule**: $Q_{4,4} \times Q_{4,4}$ yields $Q_{8,8}$. Place the binary point 8 bits from the right: 11111011.00100000.
- 4) **Verify the math**: $-1248 \times 2^{-8} = -4.875$ and $3.25 \times (-1.5) = -4.875$. The math is exact.

2) *Rescaling (Quantization)*: The hardware cannot continue doubling bit widths ($8 \rightarrow 16 \rightarrow 32 \rightarrow 64$) without exhausting logic resources. After multiplication, the $Q_{8,8}$ product must be **rescaled** to $Q_{4,4}$ before entering the next neural-network layer.

Conversion from $Q_{8,8}$ back to $Q_{4,4}$ uses an **Arithmetic Right Shift (ASR)** by four bits, dropping the lowest four fractional bits.

- Full $Q_{8,8}$: 11111011.0010 0000.
- Shift right by 4: 11111011.
- Result in $Q_{4,4}$: 1111_1011 (-5 as an integer, which is $-5 \times 0.0625 = -0.3125$. Wait! $-4.875 \neq -0.3125$!)

```

=====
<< MODULE DECOMPOSITION >>
=====
+-----+
| 1. SOFTWARE SUBSYSTEM (Processing System - ARM PS) |
| |
| [SW-01] Preprocessing & Tokenizer (OpenCV image resizing, normalization) |
| [SW-02] CNN Feature Extractor (MobileNet Inverted Residual Blocks) |
| [SW-03] Projection Layers (Linear transformation to generate Q, K, V tensors) |
| [SW-04] Quantization Wrapper (FP32 -> INT8 scaling before DMA transmission) |
| [SW-05] PYNQ DMA Controller (CMA buffer allocation, AXI DMA trigger & sync) |
| [SW-06] MLP & Classification Head (Post-attention LayerNorm, Linear, Softmax) |
+-----+
| 2. HARDWARE SUBSYSTEM (Programmable Logic - FPGA PL) |
| |
| [HW-01] AXI-Lite Slave Register Interface (Config: N, d_k, Start, Done flags) |
| [HW-02] AXI4-Stream Receiver & Input Ping-Pong BRAM Buffer (Q, K, V buffers) |
| [HW-03] Matrix Multiplier 1 (Systolic MAC Array for Q x K^T) |
| [HW-04] Attention Scaler (Arithmetic Right-Shift Divider) |
| [HW-05] Softmax Engine - Method A (Lookup Table ROM + Accumulator + Divider) |
| [HW-06] Softmax Engine - Method B (Piecewise-Linear Approximation) |
| [HW-07] Matrix Multiplier 2 (Score x V Aggregation MAC Array) |
| [HW-08] Hardware Performance Profiler (Cycle counters, stall cycle detectors) |
| [HW-09] AXI4-Stream Transmitter & Output Buffer (Streams attended tensor back) |
+-----+

```

Fig. 9: Module Decomposition.

TABLE XI: Technical comparison: Task ID; Task Description; Phase; Direct Predecessor(s); Est. Duration.

Task ID	Task Description	Phase	Direct Predecessor(s)	Est. Duration
T01	Toolchain setup (Vivado, VS Code, KV260 Ubuntu/PYNQ)	Phase 1	—	1.0 wk
T02	Literature review & mathematical formulation of Attention/Softmax	Phase 1	—	1.0 wk
T03	Train PyTorch FP32 MobileViT baseline model	Phase 2	T01, T02	1.5 wks
T04	Python INT8 quantization & fixed-point simulation	Phase 2	T03	1.5 wks
T05	Python simulation of Softmax Approximations (LUT vs. PWL)	Phase 2	T04	1.0 wk
T06	Golden vector extraction script (Generate .hex stimuli)	Phase 2	T05	0.5 wk
T07	Design MAC Processing Element & Matrix Multiply 1 (QK^T)	Phase 3	T02, T06	2.0 wks
T08	Design Scaler & Softmax Method A (LUT-based)	Phase 3	T06	1.5 wks
T09	Design Matrix Multiply 2 ($\text{Score} \times V$) & Output Logic	Phase 3	T07	1.5 wks
T10	Design AXI-Lite Regs, AXI-Stream Adapters & Ping-Pong Buffers	Phase 3	T01	1.5 wks
T11	Unit verification & self-checking testbench in Vivado xsim	Phase 4	T07, T08, T09, T10	2.0 wks
T12	RTL Pipelining, retiming & timing closure optimization ($F_{\max}$)	Phase 4	T11	1.5 wks
T13	Package custom IP & build Vivado Block Design with AXI DMA	Phase 5	T12	1.0 wk
T14	Bitstream synthesis, routing & hardware handoff (.bit/hwh)	Phase 5	T13	0.5 wk
T15	Develop PYNQ host driver & verify DMA loopback on KV260	Phase 5	T01, T14	1.0 wk
T16	Full end-to-end MobileViT deployment on KV260 (Phase 1 Complete)	Phase 5	T15	1.5 wks
T17	Implement Softmax Method B (Piecewise-Linear RTL)	Phase 6	T12	1.5 wks
T18	Add Hardware Telemetry Counters (Cycles, Stalls, DMA Wait)	Phase 6	T13	1.0 wk
T19	Deploy extended overlay & run multi-approximation benchmarks	Phase 6	T16, T17, T18	2.0 wks
T20	Performance analysis, trade-off synthesis & final capstone report	Phase 6	T19	1.5 wks

Cause of the failure: The integer part grew too large. Although -4.875 fits in $Q_{4.4}$, the naive shift truncated the

sign extension. The implementation must select the exact four integer bits, including the sign bit, and four fractional bits. From 1111_1011.0010_0000, the middle eight bits are 1011.0010. The value of 1011_0010 in $Q_{4.4}$ is $-8 + 2 + 1 + 0.125 = -4.875$.

E. 5. Critical Hardware Hazards

Implementation of the Vision Transformer MAC arrays must address two major issues:

1) *A. Rounding vs. Truncation:* When rescaling via shifting, the hardware simply cuts off the bottom bits (Truncation). Truncation always rounds *down* (towards negative infinity). Over millions of neural network multiplications, this creates a massive DC bias that ruins model accuracy.

- **Hardware Fix (Round-to-Nearest):** Before shifting right by n bits, add 2^{n-1} (which is exactly 0.5 in the shifted domain) to the accumulator. This forces standard algebraic rounding.

2) *B. Overflow and Saturation:* Standard integers wrap around. Adding 1 to the maximum positive number 0111 (+7) produces 1000 (−8). In a neural network, this discontinuity can severely corrupt the feature map.

- **Hardware Fix (Saturation):** Clipping logic clamps an accumulator above the maximum positive value to MAX_VAL and an accumulator below the minimum value to MIN_VAL.

F. 6. SystemVerilog Example: A Safe MAC Unit

The following pipeline-ready RTL example applies fixed-point arithmetic on the KV260. It multiplies two $Q_{4.4}$ inputs, adds them to an accumulator, rounds the result, and saturates it.

```

1 module fixed_point_mac (
2   input  logic signed [7:0] a,      // Q4.4 input
3   input  logic signed [7:0] b,      // Q4.4 input
4   input  logic signed [7:0] accum,  // Q4.4 previous sum
5   output logic signed [7:0] result  // Q4.4 saturated output
6 );
7
8 // 1. Multiplication: Q4.4 x Q4.4 = Q8.8 (16 bits)
9 logic signed [15:0] product;
10 assign product = a * b;
11
12 // 2. Align Accumulator to Q8.8 for addition
13 logic signed [15:0] accum_aligned;
14 assign accum_aligned = accum <<< 4; // Shift left by 4 to
    make it Q8.8
15
16 // 3. Add and apply Rounding (Add half an LSB: 2^(4-1) = 8)
17 logic signed [16:0] sum_full;      // 17 bits to catch
    overflow
18 assign sum_full = product + accum_aligned + 17'sd8;
19
20 // 4. Rescale (Shift Right) and Saturate
21 always_comb begin
22   // Shift right by 4 to return to Qx.4
23   logic signed [16:0] scaled_sum = sum_full >>> 4;
24
25   // Clamp to Q4.4 limits [-128 (0x80) to +127 (0x7F)]
26   if (scaled_sum > 17'sd127)
27     result = 8'h7F;
28   else if (scaled_sum < -17'sd128)
29     result = 8'h80;
30   else
31     result = scaled_sum[7:0];
32 end
33
34 endmodule

```

Listing 10: Source code, configuration, command sequence, or directory structure transcribed verbatim.

1) Sources:

- 1) Fixed-Point Representation: The Q Format and Addition Examples
- 2) Q (number format) - Wikipedia